DATASENTINEL · TELECOM DATA QUALITY

From Quarantine to Silver, explained so simply a child could follow

What happens to every row after the Quality Gate: auto-fix, re-check, the five monitors, and the whole Silver layer.
Line by line through the real code, every output table, every column, and how they connect, with real rows from the project.

Covers
21_quarantine_autofix · 22_revalidate · 30_observability_monitors · 49_silver_lib · 50_silver_history · 51_silver_clean · 52_silver_customer_xref · 53_silver_verify, plus the 11_dq_lib helpers they use

Data
Real numbers and rows from the Databricks catalog `data_sentinel`, read on 3 November 2026. Personal data is masked.

How to read this book

Imagine a **sorting factory for information**. Lorries (the source systems) bring boxes (rows of data). At the door stands a strict inspector, the **Quality Gate**, with a rule book (the **data contract**). Good boxes go onto the clean shelf. Broken boxes go into a **quarantine room**, a waiting room for sick boxes. This book starts the moment the inspector has finished, and follows the boxes until they are neatly stored in the **Silver** warehouse.

You will see…	It means
Code table	Every line of the real notebook, with its line number, and next to it what the line does in plain words.
Real example	Actual rows taken from the project's tables, so you can see the idea with real data.
Think of it like…	A simple picture from everyday life that explains the idea.
Watch out	Something surprising or important to remember.

Little dictionary (read once, use everywhere)

Word	Simple meaning
Table	A big grid, like a spreadsheet. Each **row** is one thing (one customer, one payment). Each **column** is one fact about it (phone, amount).
Notebook	A file of code that Databricks runs from top to bottom, like a recipe. Its name starts with a number so the order is clear (21, 22, 30…).
Dataset	One kind of data, for example `crm_customers` or `billing_payments`. Each has its own contract.
Data contract	A YAML file (`contract/*.yaml`) that says which columns a dataset has, what type they are, which rules they must follow, and how to fix simple mistakes.
Rule	One test, e.g. "the phone must look like +316 followed by 8 digits". Each rule has an **action**: *reject* (send to quarantine), *autofix* (send to quarantine to be repaired), or *flag* (let it pass with a sticker).
Quarantine	Tables in the schema `data_sentinel.quarantine` that hold rows which broke a rule, with the reason and a status.
Silver	Tables in `data_sentinel.silver` with data you can trust: only rows that passed, cleaned, with history and with personal data hidden.
Delta table	The table format Databricks uses. It can **MERGE** (update the rows that match, insert the new ones) safely, and it remembers old versions.
MERGE	Like matching name tags: "if this row already exists (same `_row_id`), change it; if not, add it".
Hash	A secret fingerprint of a value. The same phone always gives the same fingerprint, but you cannot turn the fingerprint back into the phone.
Spark / PySpark	The engine that runs the code on many computers at once. `F.col("x")` means "the column x", `F.lit(5)` means "the fixed value 5".
Run id	A label for one run of the pipeline, like `20261103163247-a1b2c3` (date and time + a random tail).

Part 0 · The big picture

Here is the journey in this book. Read it left to right; each arrow is one notebook.

Quality Gate (20) splits every row → **Auto-fix (21)** repairs simple mistakes → **Re-check (22)** tests the repairs → **Monitors (30)** checks the data as a whole → **Silver (50, 51, 52)** history, clean tables, links → **Verify (53)** checks Silver

Think of it like...

A hospital for data. The **Gate** is the doctor at the entrance. Healthy patients (good rows) go straight home (the `_validated` table). Sick patients go to the **quarantine ward**. The **Auto-fix nurse** (21) treats the easy cases with a plaster (fix the phone format). The **Re-check doctor** (22) examines the treated patients again: cured ones go home, the others stay. The **Monitors** (30) are the hospital managers who look at the big numbers: did all the ambulances arrive today? Are there suddenly ten times more patients? Finally **Silver** is the neat patient archive where every visit is kept in order and names are covered for privacy.

Which notebook writes which table

Notebook	Reads (input)	Writes (output)	How it writes

20 quality gate	Bronze tables	`silver.<ds>_validated`
		`quarantine.<ds>`
		`monitoring.dq_results`, `incidents`
			validated: overwrite (rebuilt)
			quarantine: MERGE (new in, gone out, status kept)

21 auto-fix	`quarantine.<ds>` (status PENDING_FIX)	`quarantine.<ds>` (same table!) `monitoring.dq_results`	MERGE: fills `_fixed`, changes status
22 re-check	`quarantine.<ds>` (FIXED_PENDING_REVALIDATION, REVALIDATED_OK)	`silver.<ds>_validated` (adds rows) `quarantine.<ds>` (status) `monitoring.dq_results`	MERGE insert-only into validated; MERGE update in quarantine
30 monitors	Bronze tables + `bronze.file_audit` + contracts	`monitoring.dq_results`, `monitoring.incidents`	append results; open/close incidents
49 silver lib	(nothing, only helpers)	(nothing)	—
50 history	`silver.crm_customers_validated`, `silver.crm_plans_validated`	`silver.customer_history`, `silver.plan_history`	overwrite (rebuilt every run)
51 clean	the other `*_validated` tables	`silver.account`, `invoice`, `invoice_line`, `payment`, `invoice_file`, `payment_file`, `network_cdr`, `partner_usage` (+ `app_event`)	overwrite (rebuilt every run)
52 xref	`silver.customer_history`, `silver.account`	`silver.customer_xref`	overwrite
53 verify	all Silver tables	nothing (prints PASS / WARN / FAIL)	read only

Watch out

Notebooks 21 and 22 do not create new tables. They **change the quarantine table in place** and 22 **adds rows** to the `_validated` table. That is why the same table appears as input and output.

The life of a row: the four statuses

Status in quarantine	Who sets it	Meaning for a child	What happens next
REJECTED	Gate (20) or Re-check (22)	"Broken and we cannot fix it ourselves."	Stays until the source is corrected, or a person fixes it in the review app.
PENDING_FIX	Gate (20)	"Broken, but the rule book says how to fix it."	Auto-fix (21) repairs it.
FIXED_PENDING_REVALIDATION	Auto-fix (21) or the review app	"Repaired, waiting for the doctor to check again."	Re-check (22) tests it.
REVALIDATED_OK	Re-check (22)	"Repaired and now healthy."	It is (and stays) in the `_validated` table with `_dq_fixed = true`.

Where the rows are today

dataset	status	rows
billing_accounts	REJECTED	16
crm_customers	REJECTED	52
crm_customers	REVALIDATED_OK	39
invoices_file	REJECTED	58
network_cdr	REJECTED	450
partner_usage	REJECTED	15
payments_file	REJECTED	10

Example: `crm_customers` has 91 rows in quarantine: 52 are REJECTED and 39 were repaired (REVALIDATED_OK). Together with the 940 rows that passed straight away, that makes the 979 rows in `silver.crm_customers_validated` (940 + 39), and 979 + 52 = 1,031 rows that arrived in Bronze.

Part 1 · What the Quality Gate leaves behind

Before we follow the next steps we must know the two tables the gate fills, because every later notebook reads or changes them. They are made by two small functions in `11_dq_lib`: `build_validated` and `build_quarantine`.

The shared names (11_dq_lib, lines 34–43)

The code (11_dq_lib, lines 34–43)	In plain words
`34 CATALOG = "data_sentinel"` `35 CONTRACT_DIR = f"/Volumes/{CATALOG}/monitoring/contracts"`	`CATALOG` is the name of the big cupboard in Databricks where all our tables live: `data_sentinel`. `CONTRACT_DIR` is the folder (a Databricks *Volume*) where the contract YAML files are stored.

`36`
`37 T_RESULTS = f"{CATALOG}.monitoring.dq_results"`
`38 T_INCIDENTS = f"{CATALOG}.monitoring.incidents"`
`39 T_RULES = f"{CATALOG}.monitoring.dq_rules"`
`40 T_LINEAGE = f"{CATALOG}.monitoring.lineage_edges"`
`41 T_AUDIT = f"{CATALOG}.bronze.file_audit"`
	Short names for tables used everywhere: the results log (`dq_results`), the incident list, the rule catalogue, the lineage list and the file audit (one row per uploaded file).

`42` `43 META_COLS = ["_row_id", "_file_name", "_offset", "_ts_ms", "_op", "_is_latest"]`	**META_COLS**: six technical columns that travel with every row through every table. Think of them as the **luggage label** on a suitcase: who you are (`_row_id`), which file you came from, your position in Kafka (`_offset`), when you changed (`_ts_ms`), what kind of change (`_op`) and whether you are the newest version (`_is_latest`).

build_validated, build_quarantine and upsert_quarantine (11_dq_lib, lines 408–437)

The code (11_dq_lib, lines 408–437)	In plain words
`408 def build_validated(df, c, fixed=False):` `409 cols = [x["name"] for x in c["columns"]]`	A function that shapes a **good** row for the validated table. `cols` = the column names from the contract.
`410 return df.select(*[F.col(f"t_{n}").alias(n) for n in cols], *[F.col(m) for m in META_COLS],`	Take the **typed** version of each column (`t_amount` is the amount turned into a real number) and give it back its normal name (`amount`). Add the six luggage-label columns.
`411 F.col("_flags"), F.lit(fixed).alias("_dq_fixed"),` `412 F.lit(c["version"]).alias("_contract_version"), F.current_timestamp().alias("_validated_at"))`	Add `_flags` (stickers from flag rules), `_dq_fixed` (true only if the row was repaired), the contract version and the time it was validated.
`413` `414`	(empty lines)
`415 def build_quarantine(df, c, run_id):` `416 cols = [x["name"] for x in c["columns"]]`	A function that shapes a **bad** row for quarantine.
`417 return df.select(*[F.col(n) for n in cols], *[F.col(m) for m in META_COLS],`	Keep the columns **exactly as they arrived** (raw text, not typed). That way the wrong value, e.g. `abc`, can still be seen.
`418 F.col("_failed_rules"),` `419 F.concat_ws(",", F.col("_failed_rules")).alias("_failure_reasons"),`	`_failed_rules`: the list of rules this row broke. `_failure_reasons`: the same list glued into one text with commas.
`420 F.when(F.col("_dq_status") == "REJECT", "REJECTED").otherwise("PENDING_FIX").alias("_quarantine_status"),`	The **status**: if the gate said REJECT → `REJECTED`; otherwise (it said AUTOFIX) → `PENDING_FIX`.
`421 F.lit(None).cast("map<string,string>").alias("_fixed"),` `422 F.lit(0).alias("_attempts"),`	`_fixed` starts empty (nothing repaired yet) and `_attempts` starts at 0.

`423 F.current_timestamp().alias("_quarantined_at"),`
`424 F.lit(c["version"]).alias("_contract_version"),`
`425 F.lit(run_id).alias("_run_id"))`
	When it entered quarantine, which contract version judged it, and the run id.

`426` `427`	(empty lines)
`428 def upsert_quarantine(bad_df, table):` `429 """Insert new bad rows, keep the status of rows already there, drop rows that are no longer bad."""`	**upsert_quarantine** puts the bad rows of this run into the quarantine table. Its rule in one sentence: new bad rows in, old status kept, rows that are no longer bad removed.

`430 if not spark.catalog.tableExists(table):`
`431 bad_df.write.format("delta").saveAsTable(table)`
`432 return`
	First time ever? Just create the table with these rows.

`433 (DeltaTable.forName(spark, table).alias("t")` `434 .merge(bad_df.alias("s"), "t._row_id = s._row_id")`	Otherwise MERGE by `_row_id` (the name tag).
`435 .whenNotMatchedInsertAll()`	**whenNotMatchedInsertAll**: a bad row we have never seen → add it.
`436 .whenNotMatchedBySourceDelete()` `437 .execute())`	**whenNotMatchedBySourceDelete**: a row in quarantine that is *not* bad any more in this run (the source was corrected, or the rule changed) → remove it. Rows that match are left alone, so their status, `_fixed` and `_attempts` survive between runs.

Think of it like...

The quarantine room keeps a **card per patient**. Every morning the doctor brings the new list. New names get a new card. Old cards stay as they are (with the nurse's notes). Cards of patients who are not on the list any more are thrown away: they got better at home.

Every column of the two tables

Good rows: silver.crm_customers_validated (979 rows today)

Column	Type	What it means (simple words)
customer_id	bigint	The customer's number in the CRM system (a whole number).
first_name	string	First name (raw, personal data).
last_name	string	Last name (raw, personal data).
email	string	E-mail address (raw, personal data).
msisdn	string	Mobile phone number, for example +31612345678 (raw, personal data). MSISDN is the telecom word for a phone number.
city	string	City where the customer lives.
country	string	Country code, e.g. NL.
status	string	Customer status: ACTIVE, SUSPENDED or CHURNED.
plan_id	int	Which price plan the customer has (links to crm_plans).
created_at	timestamp	When the customer was created in the CRM.
updated_at	timestamp	When the customer was last changed in the CRM.
_row_id	string	The row's name tag. For change events it is topic|partition|offset (where the message sat in Kafka); for file rows a fingerprint of the file and the row.
_file_name	string	The file the row came from (empty for change events from the database).
_offset	bigint	The position of the message in Kafka (0, 1, 2, ...).
_ts_ms	bigint	When the change happened in the source, in milliseconds since 1970.
_op	string	Kind of change: r = first snapshot, c = created, u = updated (d = deleted rows are not kept).
_is_latest	boolean	true if this is the newest event for this customer.
_flags	array<string>	Stickers: names of flag rules that failed. The row still passed. Empty list = no stickers.
_dq_fixed	boolean	true = this row only got here after a repair (auto-fix or the review app). false = it passed straight away.
_contract_version	string	Which version of the contract judged the row (e.g. 1.0 or 1.1).
_validated_at	timestamp	When the row was written here.

Bad rows: quarantine.crm_customers (91 rows today)

Column	Type	What it means (simple words)
customer_id	string	The customer's number in the CRM system (a whole number).
first_name	string	First name (raw, personal data).
last_name	string	Last name (raw, personal data).
email	string	E-mail address (raw, personal data).
msisdn	string	Mobile phone number, for example +31612345678 (raw, personal data). MSISDN is the telecom word for a phone number.
city	string	City where the customer lives.
country	string	Country code, e.g. NL.
status	string	Customer status: ACTIVE, SUSPENDED or CHURNED.
plan_id	string	Which price plan the customer has (links to crm_plans).
created_at	string	When the customer was created in the CRM.
updated_at	string	When the customer was last changed in the CRM.
_row_id	string	The row's name tag. For change events it is topic|partition|offset (where the message sat in Kafka); for file rows a fingerprint of the file and the row.
_file_name	string	The file the row came from (empty for change events from the database).
_offset	bigint	The position of the message in Kafka (0, 1, 2, ...).
_ts_ms	bigint	When the change happened in the source, in milliseconds since 1970.
_op	string	Kind of change: r = first snapshot, c = created, u = updated (d = deleted rows are not kept).
_is_latest	boolean	true if this is the newest event for this customer.
_failed_rules	array<string>	List of the rules this row broke, e.g. [CUST_MSISDN_FORMAT].
_failure_reasons	string	The same list as text. After a failed re-check it starts with autofix_failed: ...
_quarantine_status	string	REJECTED, PENDING_FIX, FIXED_PENDING_REVALIDATION or REVALIDATED_OK (see Part 0).
_fixed	map<string,string>	A small dictionary column → repaired value, filled by auto-fix (21) or the review app. Empty (null) until someone repairs the row.
_attempts	int	How many times a repair was tried (0, 1, 2, ...).
_quarantined_at	timestamp	When the row entered quarantine.
_contract_version	string	Which version of the contract judged the row (e.g. 1.0 or 1.1).

Column	Type	What it means (simple words)
`_run_id`	`string`	The pipeline run that put the row in quarantine.

Watch out

In `_validated` the values are **typed** (customer_id is a number, created_at a timestamp). In quarantine they are **all text**, exactly as received, because a broken value like `ABC` could not be turned into a number. Also: the `_validated` tables still hold raw personal data; the hiding happens in the Silver tables built by notebooks 50–52.

Part 2 · Notebook 21: the Auto-fix nurse

Job in one sentence: take every quarantined row with status PENDING_FIX, repair it using the `fixes` section of its contract, store the repaired row in `_fixed` and mark it FIXED_PENDING_REVALIDATION. It never guesses: only fixes written in the contract are used.

The rule book part it uses (contract/crm_customers.yaml)

```
fixes:
  - column: msisdn
    steps:
      - {op: trim}                                          # remove spaces at both ends
      - {op: regex_replace, pattern: "[ -]", replacement: ""}    # remove spaces and dashes inside
      - {op: regex_replace, pattern: "^0(6[0-9]{8})$", replacement: "+31$1"}   # 06xxxxxxxx  ->  +316xxxxxxxx
    examples: [["0612345678", "+31612345678"], ["06 12345678", "+31612345678"], ["06-12345678", "+31612345678"]]
```

Line by line

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run ./11_dq_lib`
 	
`# COMMAND ----------`
	Every notebook starts with these lines. Line 1 tells Databricks this file is a notebook. Lines 2–5 pick the Databricks software version (environment 6). The `# COMMAND ----------` lines cut the notebook into cells, like pages in a notebook. Line 6 (`%run ./11_dq_lib`) first runs the shared library, so all its functions (load_contracts, fix_expression, result_row…) can be used here.

`# QUARANTINE: AUTO-FIX. Repairs the rows the gate marked PENDING_FIX, using the 'fixes' section of the contract.`
`#   fixes defined      -> PENDING_FIX becomes FIXED_PENDING_REVALIDATION (fixed values are stored in _fixed)`
`#   no fixes defined   -> PENDING_FIX becomes REJECTED`
`# Run after 20_quality_gate and before 22_revalidate.`
 	
`# COMMAND ----------`
	A comment that explains the notebook: with fixes → FIXED_PENDING_REVALIDATION; without fixes → REJECTED. It must run after 20 and before 22.

`contracts = load_contracts()`	Read all contract files (one per dataset) into a dictionary: name → contract.
`run_id = new_run_id()` `results = []`	Make a new run id for this run, and an empty list where we collect result rows.
`for name, c in contracts.items():`	Go through the datasets one by one (crm_customers, billing_accounts, …). `name` = dataset name, `c` = its contract.

`    qt = quarantine_table(c)`
`    if not spark.catalog.tableExists(qt):`
`        continue`
	Find its quarantine table, e.g. `data_sentinel.quarantine.crm_customers` . If it does not exist (the dataset never had a bad row), skip to the next dataset.

`    pending = spark.table(qt).filter("_quarantine_status = 'PENDING_FIX'")` `    n = pending.count()`	Pick only the rows whose status is **PENDING_FIX** and count them.

`    if n == 0:`
`        print(f"{name}: nothing to fix")`
`        continue`
	None? Print “nothing to fix” and move on.

`    target = DeltaTable.forName(spark, qt).alias("t")`	Open the quarantine table so we can change rows in it (`DeltaTable`), and call it `t` (target).

`    if not c.get("fixes"):`
`        target.merge(pending.select("_row_id").alias("s"), "t._row_id = s._row_id AND t._quarantine_status = 'PENDING_FIX'") \`
`            .whenMatchedUpdate(set={"_quarantine_status": "'REJECTED'"}).execute()`
	If the contract has **no fixes at all**, there is nothing we can do: MERGE those rows by `_row_id` and set their status to **REJECTED**.

`        print(f"{name}: no fixes defined, {n:,} rows rejected")`
`        results.append(result_row(run_id, name, "autofix", "__AUTOFIX__", "validity", "all", n, n, "FAIL", "medium",`
`                                  "no fixes defined in the contract", c["version"]))`
`        continue`
	Print how many were rejected, write a result row (check `autofix` , FAIL, “no fixes defined in the contract”) and go to the next dataset.

`    cols = [x["name"] for x in c["columns"]]`	The contract has fixes. Get the list of all column names.

`    pairs = []`
`    for col in cols:`
`        pairs += [F.lit(col), fix_expression(col, c)]`
	Build pairs *column name → repaired value*. For each column, `fix_expression` returns the column with the contract's steps applied (trim, regex_replace…). Columns without a fix are returned unchanged.

`    upd = pending.select("_row_id", F.create_map(*pairs).alias("_fixed"))`	For each pending row, make a **map** (a little dictionary) of all columns with their repaired values. This map is the `_fixed` column.

`    (target.merge(upd.alias("s"), "t._row_id = s._row_id AND t._quarantine_status = 'PENDING_FIX'")`
`        .whenMatchedUpdate(set={"_fixed": "s._fixed",`
`                                "_quarantine_status": "'FIXED_PENDING_REVALIDATION'",`
`                                "_attempts": "t._attempts + 1"}).execute())`
	MERGE into quarantine by `_row_id` , only rows still PENDING_FIX: write `_fixed` , set status **FIXED_PENDING_REVALIDATION**, add 1 to `_attempts` .

The code (exactly as it runs in Databricks)	In plain words

`print(f"{name}: fixes applied to {n:,} rows")`
`results.append(result_row(run_id, name, "autofix", "__AUTOFIX__", "validity", "all", n, 0, "PASS", "info",`
`"fixes applied, waiting for revalidation", c["version"]))`
	Print how many rows were repaired and write a PASS result row ("fixes applied, waiting for revalidation").

`save_results(results)`	Save all result rows to `monitoring.dq_results` (append).

The helper it calls: fix_expression (11_dq_lib, lines 440–457)

The code (11_dq_lib, lines 440–457)	In plain words
`def fix_expression(name, c):` `"""Column expression for the fixed (still raw/string) value of one column."""`	Gives back the repaired value of one column, still as text.

`steps = []`
`for fx in c.get("fixes", []):`
`if fx["column"] == name:`
`steps = fx["steps"]`
	Look in the contract's `fixes` for this column and take its steps. No fix for this column → no steps.

`e = F.col(name)`	Start with the column as it is.

`for s in steps:`
`op = s["op"]`
`if op == "trim":`
`e = F.trim(e)`
`elif op == "lower":`
`e = F.lower(e)`
`elif op == "upper":`
`e = F.upper(e)`
`elif op == "regex_replace":`
`e = F.regexp_replace(e, s["pattern"], s["replacement"])`
`return e`
	Apply every step in order: **trim** (cut spaces at the ends), **lower**/**upper** (small/capital letters), **regex_replace** (find a pattern and replace it; `$1` means "the part inside the first brackets").

What it produces

Output	What changes
`quarantine.<ds>`	Rows that were PENDING_FIX: `_fixed` filled, `_quarantine_status` = FIXED_PENDING_REVALIDATION, `_attempts` + 1. Nothing else changes.
`monitoring.dq_results`	One row per dataset that had pending rows (check_name = `autofix`).

Real rows written by notebook 21 to monitoring.dq_results (latest run)

dataset	check_name	rule_id	dimension	scope	rows_checked	rows_failed	pass_rate	status	severity	detail
billing_accounts	autofix	__AUTOFIX__	validity	all	16	0	1.0	PASS	info	fixes applied, waiting for revalidation
crm_customers	autofix	__AUTOFIX__	validity	all	96	0	1.0	PASS	info	fixes applied, waiting for revalidation

Real example: one repair

Customer 1 sent an update with phone `0660****680` (local format). The gate saw it break `CUST_MSISDN_FORMAT` (an *autofix* rule) → PENDING_FIX. Notebook 21 ran the three steps: trim → nothing; remove dashes → nothing; `^0(6…)` → `+31` + the rest = `+3166****680`. It wrote that into `_fixed['msisdn']` and set FIXED_PENDING_REVALIDATION, attempts = 1.

Watch out

Auto-fix also repairs rows that cannot become correct: `ABC` stays `ABC` and `0612345` stays `0612345` (too short). Notebook 21 does not judge; that is the job of notebook 22.

Part 3 · Notebook 22: the Re-check doctor

Job in one sentence: test every repaired row again with the **same rules**; rows that now pass are added to the validated table (marked `_dq_fixed = true`) and get status REVALIDATED_OK; rows that still fail go back to REJECTED with the reason.

> **Think of it like...**
> Homework you corrected after the teacher marked it wrong. You do not decide yourself that it is right now: the teacher checks it again with the same answer sheet.

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run ./11_dq_lib`
 	
`# COMMAND ----------`
	Every notebook starts with these lines. Line 1 tells Databricks this file is a notebook. Lines 2–5 pick the Databricks software version (environment 6). The `# COMMAND ----------` lines cut the notebook into cells, like pages in a notebook. Line 6 loads the shared library.

`# REVALIDATE: run the same rules again on the repaired rows.`
`#   rows that now pass -> inserted into silver.<dataset>_validated (_dq_fixed = true), status REVALIDATED_OK`
`#   rows that still fail -> status REJECTED, with the new failed rules`
`# It is safe to run repeatedly: rows already revalidated are re-inserted if the gate rebuilt validated.`
`# Uniqueness rules are skipped here (they were tested on the original values by the gate).`
 	
`# COMMAND ----------`
	The plan: passing rows → validated with `_dq_fixed = true`; failing rows → REJECTED. Safe to run again and again. Uniqueness rules are skipped (they were already tested on the original values).

`contracts = load_contracts()`
`run_id = new_run_id()`
`results, summary = [], []`
	Read the contracts, make a run id, and prepare two empty lists: result rows and a summary for the screen.

`for name, c in contracts.items():`	Go through each dataset.

`    qt, vt = quarantine_table(c), validated_table(c)`
`    if not (spark.catalog.tableExists(qt) and spark.catalog.tableExists(vt)):`
`        continue`
	Its quarantine table and its validated table. If either does not exist, skip.

`    cand = spark.table(qt).filter("_quarantine_status IN ('FIXED_PENDING_REVALIDATION', 'REVALIDATED_OK')")` `    n = cand.count()`	Candidates = rows that are **FIXED_PENDING_REVALIDATION** (new repairs) **or REVALIDATED_OK** (old repairs). Count them.
`    if n == 0:` `        continue`	No candidates → next dataset.
`    cols = [x["name"] for x in c["columns"]]` `    raw = cand.select(*[F.col("_fixed").getItem(col).alias(col) for col in cols], *[F.col(m) for m in META_COLS])`	Rebuild each candidate as a normal row, but taking every column's value **from `_fixed`** (the repaired values), plus the luggage-label columns.
`    df, rules = apply_rules(add_typed(raw, c), c, blocked=blocked_files(c), include_unique=False)`	Run the **same machinery as the gate**: `add_typed` turns text into numbers/dates, `apply_rules` runs every contract rule (except uniqueness) and gives each row a `_dq_status` PASS / REJECT / AUTOFIX.
`    ok = df.filter(F.col("_dq_status") == "PASS")` `    bad = df.filter(F.col("_dq_status") != "PASS")`	Split into `ok` (PASS) and `bad` (anything else).

`    # 1) fixed rows that pass go to validated`
`    (DeltaTable.forName(spark, vt).alias("t")`
`        .merge(build_validated(ok, c, fixed=True).alias("s"), "t._row_id = s._row_id")`
`        .whenNotMatchedInsertAll().execute())`
	Step 1: MERGE the ok rows into the validated table by `_row_id`, **insert only** (whenNotMatchedInsertAll). `build_validated(..., fixed=True)` sets `_dq_fixed = true`. Already there? Nothing happens, so no duplicates.

`    # 2) update the quarantine status`
`    q = DeltaTable.forName(spark, qt).alias("t")`
`    q.merge(ok.select("_row_id").alias("s"), "t._row_id = s._row_id") \`
`     .whenMatchedUpdate(set={"_quarantine_status": "'REVALIDATED_OK'"}).execute()`
	Step 2a: in quarantine, set the ok rows to **REVALIDATED_OK**.

`    q = DeltaTable.forName(spark, qt).alias("t")`
`    q.merge(bad.select("_row_id", "_failed_rules").alias("s"), "t._row_id = s._row_id") \`
`     .whenMatchedUpdate(set={"_quarantine_status": "'REJECTED'", "_failed_rules": "s._failed_rules",`
`                            "_failure_reasons": "concat('autofix_failed:', concat_ws(',', s._failed_rules))"}).execute()`
	Step 2b: set the bad rows to **REJECTED**, store the rules they broke now, and write the reason as `autofix_failed:RULE1,RULE2`.

`    n_ok = spark.table(qt).filter("_quarantine_status = 'REVALIDATED_OK'").count()`
`    n_rej = spark.table(qt).filter("_quarantine_status = 'REJECTED'").count()`
`    n_pend = spark.table(qt).filter("_quarantine_status IN ('PENDING_FIX', 'FIXED_PENDING_REVALIDATION')").count()`
`    n_val = spark.table(vt).count()`
`    effective = n_val / (n_val + n_rej + n_pend) if (n_val + n_rej + n_pend) else 1.0`
	Count, after the changes: how many are OK, rejected, still pending, and how many rows the validated table has. `effective` = the pass rate *after* fixing = validated ÷ (validated + rejected + pending).

`    results.append(result_row(run_id, name, "revalidate", "__REVALIDATE__", "validity", "all", n, n - n_ok,`
`                              "PASS" if n_ok == n else "WARN", "info" if n_ok == n else "medium",`
`                              f"{n_ok:,} of {n:,} repaired rows now pass", c["version"]))`
	Result row 1 (`revalidate`): how many repaired rows now pass. PASS if all of them, else WARN.

The code (exactly as it runs in Databricks)	In plain words

`59 results.append(result_row(run_id, name, "after_fix_pass_rate", "__AFTER_FIX__", "validity", "all",`
`60 n_val + n_rej + n_pend, n_rej + n_pend, "PASS" if effective >= c.get("slo", {}).get("min_pass_rate", 0.95) else "FAIL",`
`61 "info", f"validated {n_val:,}; rejected {n_rej:,}; pending {n_pend:,}", c["version"]))`
	Result row 2 (`after_fix_pass_rate`): compares the pass rate after fixing with the contract's `min_pass_rate` (SLO). PASS if high enough, else FAIL.

`62 summary.append((name, n, n_ok, n - n_ok, round(effective, 4)))` `63 print(f"{name}: {n_ok:,} of {n:,} repaired rows pass revalidation")`	Add a line to the summary and print it.

`65 save_results(results)`
`66 if summary:`
`67 display(spark.createDataFrame(summary, "dataset string, repaired long, now_pass long, still_failing long, "`
`68 "pass_rate_after_fix double").orderBy("dataset"))`
	Save the result rows to `monitoring.dq_results` and show the summary table on screen.

Why REVALIDATED_OK rows are re-checked every run

The gate (20) **rebuilds** the validated table from Bronze every run (overwrite). The repaired values are not in Bronze, so they would disappear. Re-check puts them back each time (lines 26 and 37–40). Because the MERGE is insert-only by `_row_id`, running it twice never makes duplicates.

Real results

monitoring.dq_results, check revalidate (latest run)

dataset	check_name	rule_id	dimension	scope	rows_checked	rows_failed	pass_rate	status	severity	detail
crm_customers	revalidate	__REVALIDATE__	validity	all	40	1	0.975	WARN	medium	39 of 40 repaired rows now pass

40 repaired customers were checked; 39 now pass; 1 still fails (customer 11: phone fixed, but the e-mail is still wrong).

monitoring.dq_results, check after_fix_pass_rate (latest run)

dataset	check_name	rule_id	dimension	scope	rows_checked	rows_failed	pass_rate	status	severity	detail
crm_customers	after_fix_pass_rate	__AFTER_FIX__	validity	all	1031	52	0.9496	FAIL	info	validated 979; rejected 52; pending 0

1,031 customer rows in total; 979 are validated, 52 are still rejected → 94.96%, below the 97% target, so FAIL. This is honest: repairs helped (it was 91.2% before), but 52 rows still need a person.

Real example: three customers, three endings

Two customers whose repair could not work (quarantine.crm_customers)

customer_id	msisdn	fixed_msisdn	_failed_rules	_failure_reasons	_quarantine_status	_attempts
110	0612345	0612345	['CUST_MSISDN_FORMAT']	autofix_failed:CUST_MSISDN_FORMAT	REJECTED	1
137	ABC	ABC	['CUST_MSISDN_FORMAT']	autofix_failed:CUST_MSISDN_FORMAT	REJECTED	1

customer	original phone	after auto-fix (21)	re-check (22)	final status
1	`0660****680`	`+3166****680`	passes every rule	**REVALIDATED_OK** → now in validated with `_dq_fixed = true`
110	`0612345`	`0612345` (pattern did not match: too few digits)	fails CUST_MSISDN_FORMAT	**REJECTED**, reason `autofix_failed:CUST_MSISDN_FORMAT`
137	`ABC`	`ABC`	fails CUST_MSISDN_FORMAT	**REJECTED**

The review app uses the same road

When a data owner edits a row in the Streamlit review app, the app writes the corrected row into `_fixed` and sets FIXED_PENDING_REVALIDATION, exactly like notebook 21. On the next run notebook 22 checks it. So a person cannot push a wrong value into Silver.

Part 4 · The logbook and the alarm: dq_results and incidents

Notebooks 21, 22 and 30 all write to the same **logbook** (`monitoring.dq_results`), and 30 also rings the **alarm** (`monitoring.incidents`). Three small helpers in `11_dq_lib` do this.

The code (11_dq_lib, lines 482–520)	In plain words

`482 def result_row(run_id, dataset, check, rule_id, dimension, scope, checked, failed, status, severity, detail, version):`
`483 rate = 1.0 if not checked else max(0.0, 1.0 - float(failed) / float(checked))`
`484 return (run_id, datetime.utcnow(), dataset, check, rule_id, dimension, scope, int(checked), int(failed),`
`485 float(rate), status, severity, detail, version)`
	result_row makes one logbook line. It computes `pass_rate` = 1 − failed ÷ checked (and 1.0 when nothing was checked) and stamps the current time.

`486` `487`	(empty lines)

`488 def save_results(rows):`
`489 if rows:`
`490 spark.createDataFrame(rows, RESULT_SCHEMA).write.mode("append").saveAsTable(T_RESULTS)`
	save_results appends all the lines to `monitoring.dq_results`. Append = add at the bottom; old lines are never changed, so the history of every run stays.

`491` `492`	(empty lines)

`493 def sync_incidents(run_id, failing, checked):`
`494 """failing: list of dicts(dataset, check, scope, pillar, severity, description, impacted).`
`495 checked: set of (dataset, check) pairs that were evaluated in this run.`
`496 Opens an incident for each new failure and resolves open incidents that no longer fail."""`
	sync_incidents: given the failing checks of this run and the list of checks that were done, open new incidents and close the ones that are fixed.

`497 def key(d):`
`498 return f"{d['dataset']}|{d['check']}|{d['scope']}"`
`499`
	Each incident has a key: `dataset|check|scope`, e.g. `network_cdr|freshness_missing_file|2026-10-02`. Same problem = same key.

`500 open_keys = {r["incident_key"] for r in spark.table(T_INCIDENTS).filter("status = 'OPEN'")`
`501 .select("incident_key").collect()}`
`502 fail_keys = {key(d) for d in failing}`
	Collect the keys of incidents that are already OPEN, and the keys failing now.

`503 seen, new_rows = set(), []`
`504 for d in failing:`
`505 k = key(d)`
`506 if k in open_keys or k in seen:`
`507 continue`
`508 seen.add(k)`
`509 new_rows.append((uuid.uuid4().hex[:12], k, datetime.utcnow(), None, "OPEN", d["dataset"], d["check"],`
`510 d["pillar"], d["severity"], d["scope"], d["description"], d.get("impacted", ""),`
`511 run_id, None))`
	For each failing check whose key is not open yet: create a new incident row (random 12-character id, OPEN, time now, description, impacted tables).

`512 if new_rows:` `513 spark.createDataFrame(new_rows, INCIDENT_SCHEMA).write.mode("append").saveAsTable(T_INCIDENTS)`	Append the new incidents to `monitoring.incidents`.
`514 to_resolve = [k for k in open_keys` `515 if tuple(k.split("|", 2)[:2]) in checked and k not in fail_keys]`	Which open incidents can be closed? Those whose check **was run this time** and is **not failing any more**.

`516 if to_resolve:`
`517 quoted = ",".join("'" + k.replace("'", "''") + "'" for k in to_resolve)`
`518 spark.sql(f"UPDATE {T_INCIDENTS} SET status = 'RESOLVED', closed_at = current_timestamp(), "`
`519 f"resolved_run_id = '{run_id}' WHERE incident_key IN ({quoted})")`
	Mark them RESOLVED with the close time and the run id that proved the fix. Nobody clicks "close": the data proves it.

`520 return len(new_rows), len(to_resolve)`	Return how many were opened and how many resolved.

The logbook: monitoring.dq_results (3,936 rows today)

Column	Type	What it means (simple words)
`run_id`	`string`	Which run wrote the line.
`run_ts`	`timestamp`	When it was written.
`dataset`	`string`	Which dataset was checked.
`check_name`	`string`	Kind of check: gate_rule, gate_pass_rate, autofix, revalidate, after_fix_pass_rate, freshness_lag, freshness_missing_file, volume, distribution_..., schema_..., lineage_declared, recon_...
`rule_id`	`string`	The rule or monitor id, e.g. CUST_MSISDN_FORMAT, __AUTOFIX__, FRESHNESS_LAG.
`dimension`	`string`	Quality dimension or pillar: validity, completeness, freshness, volume, distribution, schema, lineage...
`scope`	`string`	What exactly was checked: all, latest, a file name, a date.
`rows_checked`	`bigint`	How many things were checked.
`rows_failed`	`bigint`	How many failed.
`pass_rate`	`double`	1 − failed ÷ checked (between 0 and 1).
`status`	`string`	PASS, FAIL or WARN.
`severity`	`string`	info, low, medium, high or critical.

Column	Type	What it means (simple words)
`detail`	`string`	A sentence a person can read.
`contract_version`	`string`	The contract version used.

The alarm list: `monitoring.incidents` (36 rows today)

Column	Type	What it means (simple words)
`incident_id`	`string`	Short random id of the incident.
`incident_key`	`string`	dataset|check|scope: the same problem always has the same key, so it is never opened twice.
`opened_at`	`timestamp`	When it was opened.
`closed_at`	`timestamp`	When it was closed (empty while open).
`status`	`string`	OPEN or RESOLVED.
`dataset`	`string`	Where the problem is.
`check_name`	`string`	Which check failed.
`pillar`	`string`	The kind of problem (freshness, validity, consistency…).
`severity`	`string`	How bad: low, medium, high, critical.
`scope`	`string`	Which file, date or 'all'.
`description`	`string`	The detail text of the failing check.
`impacted`	`string`	Tables downstream that are affected (from the lineage in the contracts).
`run_id`	`string`	The run that opened it.
`resolved_run_id`	`string`	The run that closed it.

Real incidents

incident_id	incident_key	opened_at	closed_at	status	dataset	check_name	pillar	severity	scope	description	impacted
037a295f70a9	network_cdr|freshness_missing_file|2026-10-02	2026-10-10 11:42:52.889926	null	OPEN	network_cdr	freshness_missing_file	freshness	high	2026-10-02	no daily file for 2026-10-02	data_sentinel.silver.fact_usage
1fce59fcce44	crm_customers|pass_rate_slo|all	2026-10-10 18:46:57.924971	null	OPEN	crm_customers	pass_rate_slo	validity	high	all	Pass rate 87.3% is below the SLO of 97%	data_sentinel.silver.customer_xref, data_sentinel.silver.dim_customer

Watch out

The second incident lists `silver.dim_customer` as impacted. That table name comes from an old lineage line in the contract; dimensions now live in Gold (`gold.dim_customer`). The incident itself is real: customers pass at 87.3% before repairs, under the 97% target.

Part 5 · Notebook 30: the Monitors (five pillars)

Job in one sentence: look at each dataset **as a whole** (not row by row) and answer five questions; every answer is a logbook line, every "no" opens an incident.

Pillar	The question	How it knows
Freshness	Did today's data arrive? Is a file missing? Is it too old?	File dates in `bronze.file_audit`; for change streams the newest Kafka timestamp.
Volume	Is the number of rows normal?	Rows per file compared with minimum, maximum and the median of all files.
Distribution	Are the values normal (e.g. average data use, drop rate)?	A number per file compared with the median of all files.
Schema	Did a column disappear or a new one appear?	Column list per file (or JSON keys) compared with the contract.
Lineage	Do we know what this data feeds?	The `lineage` section of the contract.

> **Think of it like...**
> A school bus manager. Freshness: did every bus arrive today? Volume: are there about as many children as usual, not 5 instead of 50? Distribution: is the average age normal, not suddenly 40? Schema: does every child still have a name tag? Lineage: which class does each bus bring children to?

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run ./11_dq_lib`
 	
`# COMMAND ----------`
	Every notebook starts with these lines. Line 1 tells Databricks this file is a notebook. Lines 2–5 pick the Databricks software version (environment 6). The `# COMMAND ----------` lines cut the notebook into cells, like pages in a notebook. Line 6 loads the shared library.

`# OBSERVABILITY MONITORS: the five pillars of data observability, driven by the 'monitors' section of each contract.`
`#  freshness  did the data arrive on time, and is a file missing?`
`#  volume  is the row count in the normal range?`
`#  distribution are the values in the normal range?`
`#  schema  did columns appear, disappear or change?`
`#  lineage  which tables are downstream, so an incident shows what it affects`
`# Results go to monitoring.dq_results. Failures open incidents; incidents resolve themselves when the check passes again.`
 	
`# COMMAND ----------`
	The plan: five pillars, driven by the `monitors` part of each contract; results to the logbook; failures open incidents that close themselves later.

`dbutils.widgets.text("as_of", str(date.today()))` `AS_OF = date.fromisoformat(dbutils.widgets.get("as_of"))`	A widget is a small input box. `as_of` = "pretend today is...", by default the real date. All freshness checks are measured against it.

`contracts = load_contracts()`
`edges = edges_from_contracts(contracts)`
`run_id = new_run_id()`
`rows, failing, checked = [], [], set()`
`audit_rows = spark.table(T_AUDIT).collect() if spark.catalog.tableExists(T_AUDIT) else []`
	Read the contracts and the lineage edges (who feeds whom), make a run id, prepare empty lists (logbook lines, failures, checks done), and read every row of `bronze.file_audit` (one row per uploaded file).

`def record(c, check, rule_id, pillar, scope, n_checked, n_failed, severity, detail):`
`  ok = n_failed == 0`
`  rows.append(result_row(run_id, c["dataset"], check, rule_id, pillar, scope, n_checked, n_failed,`
`  "PASS" if ok else "FAIL", "info" if ok else severity, detail, c["version"]))`
`  checked.add((c["dataset"], check))`
	record() = write one logbook line. ok when nothing failed. It remembers that this check was done (needed to close old incidents).

`  if not ok:`
`  failing.append(dict(dataset=c["dataset"], check=check, scope=scope, pillar=pillar, severity=severity,`
`  description=detail,`
`  impacted=", ".join(impacted_tables(validated_table(c), edges))))`
	If it failed, also add it to the `failing` list with the impacted tables (everything downstream of this dataset's validated table).

`def month_range(a, b):`
`  out, y, m = [], a.year, a.month`
`  while (y, m) <= (b.year, b.month):`
`  out.append(date(y, m, 1))`
`  m += 1`
`  if m > 12:`
`  m, y = 1, y + 1`
`  return out`
	month_range: all first-days-of-month between two dates (for monthly files).

`def month_end(d):`
`  return date(d.year + (d.month == 12), d.month % 12 + 1, 1) - timedelta(days=1)`
 	
`# COMMAND ----------`
	month_end: the last day of a month (first day of next month minus one day).

`for name, c in contracts.items():`
`  mon, kind, src = c.get("monitors", {}), c["source"]["kind"], c["source"]`
`  names = [x["name"] for x in c["columns"]]`
`  files = [r for r in audit_rows if kind == "csv" and r["source"] == src["file_source"]]`
	For each dataset: its monitor settings, its source kind (csv, cdc, json), its columns, and its files from the audit table.

The code (exactly as it runs in Databricks)	In plain words

`# ---------- FRESHNESS ----------`
`fr = mon.get("freshness")`
`if fr and kind == "csv":`
`  dates = sorted({r["file_date"] for r in files if r["file_date"]})`
`  if not dates:`
`  record(c, "freshness_missing_file", "FRESHNESS_MISSING_FILE", "freshness", "all", 0, 1, "high", "no files loaded yet")`
	FRESHNESS for files: take all file dates. No file at all? → fail "no files loaded yet".

`  else:`
`  if fr["frequency"] == "daily":`
`  expected = [dates[0] + timedelta(days=i) for i in range((dates[-1] - dates[0]).days + 1)]`
`  latest_end = dates[-1]`
`  else:`
`  expected = month_range(dates[0], dates[-1])`
`  latest_end = month_end(dates[-1])`
	Build the list of **expected** dates: every day between the first and the last file (daily), or every month (monthly).

`  missing = [d for d in expected if d not in set(dates)]`
`  for d in missing:`
`  record(c, "freshness_missing_file", "FRESHNESS_MISSING_FILE", "freshness", str(d), len(expected), 1, "high",`
`  f"no {fr['frequency']} file for {d}")`
	Each expected date without a file → one failing line "no daily file for 2026-10-02".

`  if not missing:`
`  record(c, "freshness_missing_file", "FRESHNESS_MISSING_FILE", "freshness", "all", len(expected), 0, "high",`
`  f"all {len(expected)} expected files present")`
	No gaps → one passing line "all N expected files present".

`  lag = (AS_OF - latest_end).days`
`  record(c, "freshness_lag", "FRESHNESS_LAG", "freshness", "latest", 1, int(lag > fr["max_delay_days"]), "high",`
`  f"latest file covers {latest_end}, {lag} days before {AS_OF} (SLA {fr['max_delay_days']} days)")`
	Lag: how many days between the newest file and today. More than `max_delay_days` → fail.

`elif fr and fr.get("type") == "stream":`
`  q = spark.table(src["table"])`
`  if kind == "cdc":`
`  q = q.filter(F.col("topic") == src["topic"])`
`  mx = q.agg(F.max("kafka_timestamp_ms")).first()[0]`
`  if mx is None:`
`  record(c, "freshness_lag", "FRESHNESS_LAG", "freshness", "latest", 1, 1, "high", "no events loaded")`
`  else:`
`  lag_min = (datetime.utcnow().timestamp() * 1000 - mx) / 60000`
`  record(c, "freshness_lag", "FRESHNESS_LAG", "freshness", "latest", 1, int(lag_min > fr["max_delay_minutes"]), "medium",`
`  f"newest event is {lag_min:,.0f} minutes old (SLA {fr['max_delay_minutes']} minutes)")`
	FRESHNESS for streams (change events): find the newest Kafka timestamp for this topic; how many minutes old is it? More than `max_delay_minutes` → fail.

`# ---------- VOLUME ----------`
`vol = mon.get("volume")`
`if vol and kind == "csv" and files:`
`  counts = {r["file_name"]: r["row_count"] for r in files}`
`  med = statistics.median(counts.values())`
`  for fn, n in sorted(counts.items()):`
`  problems = []`
`  if "min_rows" in vol and n < vol["min_rows"]:`
`  problems.append(f"below minimum {vol['min_rows']:,}")`
`  if "max_rows" in vol and n > vol["max_rows"]:`
`  problems.append(f"above maximum {vol['max_rows']:,}")`
`  if "ratio_to_median" in vol and med:`
`  lo, hi = vol["ratio_to_median"]`
`  if not (lo <= n / med <= hi):`
`  problems.append(f"{n / med:.2f}x the median, allowed {lo} to {hi}")`
`  record(c, "volume", "VOLUME_ROWS", "volume", fn, n, int(bool(problems)), "high",`
`  f"{n:,} rows (median {med:,.0f})" + (": " + "; ".join(problems) if problems else ""))`
	VOLUME for files: rows per file and the median. Too few, too many, or too far from the median (ratio outside the allowed range) → fail, with the reason in the detail.

`elif vol and kind in ("json", "cdc"):`
`  q = spark.table(src["table"])`
`  if kind == "cdc":`
`  q = q.filter(F.col("topic") == src["topic"])`
`  n = q.count()`
`  record(c, "volume", "VOLUME_ROWS", "volume", "all", n, int(n < vol.get("min_rows", 1)), "high", f"{n:,} events loaded")`
	VOLUME for streams: count all events of the topic; fewer than `min_rows` → fail.

`# ---------- DISTRIBUTION ----------`
`for m in mon.get("distribution", []):`
`  if kind != "csv":`
`  continue`
`  where = f"WHERE {m['where']} " if m.get("where") else ""`
`  q = (f"SELECT regexp_extract(_source_file, '[^/]+$', 0) AS file_name, {m['expr']} AS v "`
`  f"FROM {src['table']} {where}GROUP BY 1")`
`  vals = {r["file_name"]: r["v"] for r in spark.sql(q).collect() if r["v"] is not None}`
	DISTRIBUTION (files only): for every monitor in the contract, compute one number per file with a small SQL expression (e.g. the drop rate).

`  if len(vals) < 2:`
`  continue`
`  med = statistics.median(vals.values())`
	Need at least 2 files to compare; take the median.

The code (exactly as it runs in Databricks)	In plain words
132 for fn, v in sorted(vals.items()): 133 if "ratio_to_median" in m and med: 134 lo, hi = m["ratio_to_median"] 135 bad = not (lo <= v / med <= hi) 136 detail = f"{m['name']} = {v:,.3f} ({v / med:.2f}x the median {med:,.3f}, allowed {lo} to {hi})" 137 else: 138 tol = m["abs_diff_to_median"] 139 bad = abs(v - med) > tol 140 detail = f"{m['name']} = {v:,.3f} (median {med:,.3f}, allowed difference {tol})" 141 record(c, f"distribution_{m['name']}", f"DIST_{m['name'].upper()}", "distribution", fn, 1, int(bad), "medium", detail)	Each file's number is compared with the median: either as a ratio (must be between lo and hi) or as an absolute difference (must be ≤ tol). Outside → fail.
143 # ---------- SCHEMA ---------- 144 if kind == "csv": 145 required = [x["name"] for x in c["columns"] if x.get("required")] 146 for r in files: 147 have = [x for x in (r["columns"] or "").split(",") if x] 148 missing = [x for x in required if x not in have] 149 extra = [x for x in have if x not in names] 150 record(c, "schema_missing_required", "SCHEMA_MISSING_COLUMN", "schema", r["file_name"], len(required), len(missing), 151 "critical", f"missing required: {', '.join(missing)}" if missing else "all required columns present") 152 record(c, "schema_extra_columns", "SCHEMA_EXTRA_COLUMN", "schema", r["file_name"], len(have), len(extra), "medium", 153 f"new columns not in the contract: {', '.join(extra)}" if extra else "no new columns")	**SCHEMA for files**: compare the file's column list with the contract: missing required columns (critical) and extra columns not in the contract (medium).
154 elif kind in ("json", "cdc"): 155 q = spark.table(src["table"]) 156 path = "value" if kind == "json" else "coalesce(get_json_object(value, '$.payload.after'), get_json_object(value, '$.after'))" 157 if kind == "cdc": 158 q = q.filter(F.col("topic") == src["topic"]) 159 q.createOrReplaceTempView("_schema_probe") 160 keys = {r["k"] for r in spark.sql(f"SELECT DISTINCT explode(json_object_keys({path})) AS k FROM _schema_probe WHERE {path} IS NOT NULL").collect()} 161 extra = sorted(keys - set(names)) 162 missing = sorted(set(x["name"] for x in c["columns"] if x.get("required")) - keys) 163 record(c, "schema_extra_columns", "SCHEMA_EXTRA_COLUMN", "schema", "all", len(keys), len(extra), "medium", 164 f"new fields not in the contract: {', '.join(extra)}" if extra else "no new fields") 165 record(c, "schema_missing_required", "SCHEMA_MISSING_COLUMN", "schema", "all", len(names), len(missing), "critical", 166 f"required fields never seen: {', '.join(missing)}" if missing else "all required fields seen")	**SCHEMA for streams**: list every JSON key that appears in the messages and compare it with the contract: new fields and required fields never seen.
168 # ---------- LINEAGE ---------- 169 has_lineage = bool(c.get("lineage", {}).get("upstream")) and bool(c.get("lineage", {}).get("downstream")) 170 record(c, "lineage_declared", "LINEAGE_DECLARED", "lineage", "all", 1, int(not has_lineage), "low", 171 "upstream and downstream tables declared in the contract" if has_lineage else "no lineage declared")	**LINEAGE**: does the contract declare upstream and downstream tables? If not → low-severity fail.
173 # COMMAND ---------- 174 175 save_results(rows) 176 opened, resolved = sync_incidents(run_id, failing, checked) 177 print(f"run {run_id}: {len(rows)} checks, {len(failing)} failing, incidents opened {opened}, resolved {resolved}")	Save the logbook lines, open/close incidents with `sync_incidents`, and print a summary.
178 display(spark.createDataFrame([(r[2], r[3], r[6], r[10], r[11], r[12]) for r in rows if r[10] == "FAIL"], 179 "dataset string, check_name string, scope string, status string, severity string, detail string") 180 .orderBy("dataset", "check_name", "scope"))	Show a table of only the failing checks on screen.

Real results (one row per monitor type)

monitoring.dq_results written by notebook 30

dataset	check_name	rule_id	dimension	scope	rows_checked	rows_failed	pass_rate	status	severity	detail
network_cdr	distribution_drop_rate	DIST_DROP_RATE	distribution	cdr_20261004.csv	1	1	0.0	FAIL	medium	drop_rate = 0.403 (median 0.055, allowed difference 0.15)
payments_file	freshness_lag	FRESHNESS_LAG	freshness	latest	1	1	0.0	FAIL	high	latest file covers 2026-09-30, 34 days before 2026-11-03 (SLA 10 days)
network_cdr	freshness_missing_file	FRESHNESS_MISSING_FILE	freshness	2026-10-02	10	1	0.9	FAIL	high	no daily file for 2026-10-02
payments_file	lineage_declared	LINEAGE_DECLARED	lineage	all	1	0	1.0	PASS	info	upstream and downstream tables declared in the contract
network_cdr	schema_extra_columns	SCHEMA_EXTRA_COLUMN	schema	cdr_20261007.csv	9	1	0.8889	FAIL	medium	new columns not in the contract: roaming_flag
network_cdr	schema_missing_required	SCHEMA_MISSING_COLUMN	schema	cdr_20261006.csv	8	1	0.875	FAIL	critical	missing required: cell_id
network_cdr	volume	VOLUME_ROWS	volume	cdr_20261003.csv	500	1	0.998	FAIL	high	500 rows (median 5,000): below minimum 1,000; 0.10x the median, allowed 0.5 to 2.0

What the line says	Why it matters
network_cdr: **no daily file for 2026-10-02**	One day of network usage is missing, so usage and billing for that day are incomplete.
payments_file: **latest file covers 2026-09-30, 34 days before 2026-11-03**	The monthly payment export is late. Dashboards show old payment data.
network_cdr: **drop_rate = 0.403 (median 0.055)** for one file	40% dropped calls in one file vs 5.5% normally: either a network problem or a broken file.

Part 6 · Notebook 49: the Silver toolbox

Job in one sentence: give the Silver notebooks the same names and tools: table names, the secret salt, “newest row per key”, hashing and masking personal data, building history (SCD2) and saving a table. It writes nothing itself.

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
	Notebook header (see Part 2).

`# MAGIC %run /Workspace/Users/ahme63111@gmail.com/sentinel-dq-telecom/11_dq_lib`
 	
`# COMMAND ----------`
	Run the quality library first (absolute path to `11_dq_lib`), so CATALOG, load_contracts and validated_table exist here too.

`# Shared helpers for the Silver layer (it builds on 11_dq_lib: CATALOG, load_contracts, validated_table).`
`# Silver = rows that passed the quality gate, plus three things only: SCD2 history, PII masking, customer_xref.`
`# Dimensions, facts and marts are built in Gold by dbt, not here.`
	What Silver is: rows that passed, plus three things only: SCD2 history, PII masking, customer_xref. Stars and facts are Gold's job (dbt).

`SILVER = f"{CATALOG}.silver"`
`T_CUSTOMER_HISTORY = f"{SILVER}.customer_history"`
`T_PLAN_HISTORY = f"{SILVER}.plan_history"`
`T_XREF = f"{SILVER}.customer_xref"`
`T_ACCOUNT = f"{SILVER}.account"`
`T_INVOICE = f"{SILVER}.invoice"`
`T_INVOICE_LINE = f"{SILVER}.invoice_line"`
`T_PAYMENT = f"{SILVER}.payment"`
`T_INVOICE_FILE = f"{SILVER}.invoice_file"`
`T_PAYMENT_FILE = f"{SILVER}.payment_file"`
`T_NETWORK_CDR = f"{SILVER}.network_cdr"`
`T_APP_EVENT = f"{SILVER}.app_event"`
`T_PARTNER_USAGE = f"{SILVER}.partner_usage"`
	The full names of all Silver tables, so every notebook uses exactly the same names.

`SILVER_TABLES = [T_CUSTOMER_HISTORY, T_PLAN_HISTORY, T_XREF, T_ACCOUNT, T_INVOICE, T_INVOICE_LINE, T_PAYMENT,` `T_INVOICE_FILE, T_PAYMENT_FILE, T_NETWORK_CDR, T_APP_EVENT, T_PARTNER_USAGE]`	The list of Silver tables built here (used by the verify notebook).

`# tables of the first Silver version (dimensions and facts); they now belong to Gold (dbt) and are dropped by notebook 50`
`OLD_TABLES = [f"{SILVER}.{n}" for n in ("dim_customer", "dim_plan", "fact_usage", "fact_invoice", "fact_payment",`
`"fact_app_events", "fact_partner_usage")]`
	Names of very old Silver tables (dim_*, fact_*). Notebook 50 deletes them if they still exist, because these now belong to Gold.

`END_OF_TIME = "9999-12-31 00:00:00"`	**END_OF_TIME** = 9999-12-31. The current version of a customer is “valid until the end of time”.

`# The salt must be the same in every notebook and must not be in the code in production.`
`# Create the secret once:  databricks secrets create-scope sentinel ; databricks secrets put-secret sentinel pii_salt`
`try:`
`    PII_SALT = dbutils.secrets.get("sentinel", "pii_salt")`
`except Exception:`
`    PII_SALT = "dev-salt-change-me"`
	The **salt**: a secret word mixed into every fingerprint so nobody can rebuild phone numbers from the hashes. It is read from the Databricks secret `sentinel/pii_salt`; if that secret does not exist, a development salt is used.

`CONTRACTS = load_contracts()`	Read the contracts once.

`def validated(dataset):`
`    """The rows of a dataset that passed the quality gate (including the repaired ones)."""`
`    return spark.table(validated_table(CONTRACTS[dataset]))`
	validated(ds) = the validated table of a dataset (good rows, including repaired ones).

`def latest_per_key(df, key):`
`    """One row per business key: the newest change event (CDC) or the newest file row."""`
`    w = Window.partitionBy(*key).orderBy(F.col("_ts_ms").desc_nulls_last(), F.col("_offset").desc_nulls_last(),`
`F.col("_file_name").desc_nulls_last())`
`    return df.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1").drop("_rn")`
	latest_per_key: for each key (e.g. each payment_id) keep only the newest row: newest change time first, then the highest Kafka offset, then the newest file name. It numbers the rows 1, 2, 3… per key and keeps number 1.

`# ---------------- PII ----------------`
`def hash_pii(col, lower=False):`
`    v = F.trim(F.col(col))`
`    v = F.lower(v) if lower else v`
`    return F.sha2(F.concat(F.lit(PII_SALT), v), 256)`
	hash_pii: trim the value (and make it lower-case for e-mails), glue the salt in front, and compute SHA-256: a 64-character fingerprint.

`def mask_msisdn(col):      # +316****5678` `    return F.concat(F.substring(F.col(col), 1, 4), F.lit("****"), F.substring(F.col(col), -2, 2))`	**mask_msisdn**: first 4 characters + **** + last 2 → `+316****80`. Good for showing on screen.
`def mask_email(col):       # l***@example.nl` `    return F.concat(F.substring(F.col(col), 1, 1), F.lit("***@"), F.substring_index(F.col(col), "@", -1))`	**mask_email**: first letter + ***@ + the domain → `m***@example.nl`.
`def mask_name(col):        # L***` `    return F.concat(F.substring(F.col(col), 1, 1), F.lit("***"))`	**mask_name**: first letter + *** → `S***`.

The code (exactly as it runs in Databricks)	In plain words

`# ---------- SCD2 ----------`
`def scd2(events, key, tracked, created_col=None):`
`  """Turn ordered change events into history rows.`
`  events : one row per change event, with _ts_ms and _offset (CDC metadata)`
`  key  : business key column name`
`  tracked: columns whose change starts a new version`
`  A new version starts only when a tracked column really changes.`
`  valid_from of the first version is the creation time (when known), then the event time.`
`  valid_to is the next valid_from, or END_OF_TIME for the current version."""`
	scd2(): turns a list of change events into a history with versions. Its explanation (comment) is in the code itself; the next lines do it.

`  w = Window.partitionBy(key).orderBy(F.col("_ts_ms").asc(), F.col("_offset").asc())`	Order the events of each customer by time (then offset).
`  h = F.sha2(F.concat_ws("||", *[F.coalesce(F.col(c).cast("string"), F.lit("~null~")) for c in tracked]), 256)` `  x = events.withColumn("_attr_hash", h)`	Make a fingerprint of the **tracked** columns of each event (e.g. phone hash, city, plan). Two events with the same fingerprint have the same tracked values.
`  x = x.withColumn("_prev_hash", F.lag("_attr_hash").over(w))` `  x = x.filter(F.col("_prev_hash").isNull() | (F.col("_prev_hash") != F.col("_attr_hash")))`	Compare each event with the previous one. Keep an event only if it is the first, or if its fingerprint is **different** (something tracked really changed). Repeats are dropped.
`  event_ts = F.expr("timestamp_millis(_ts_ms)")` `  x = x.withColumn("version", F.row_number().over(w))`	The kept events become versions 1, 2, 3… in time order.
`  first_from = event_ts if created_col is None else F.least(F.coalesce(F.col(created_col), event_ts), event_ts)` `  x = x.withColumn("valid_from", F.when(F.col("version") == 1, first_from).otherwise(event_ts))`	**valid_from**: version 1 starts at the creation time (or the event time if that is earlier/unknown); later versions start at their event time.

`  w2 = Window.partitionBy(key).orderBy(F.col("version").asc())`
`  x = x.withColumn("valid_to", F.coalesce(F.lead("valid_from").over(w2), F.lit(END_OF_TIME).cast("timestamp")))`
`  x = x.withColumn("is_current", F.col("valid_to") == F.lit(END_OF_TIME).cast("timestamp"))`
	valid_to = the start of the next version; the last version gets END_OF_TIME and **is_current** = true.

`  return x.drop("_attr_hash", "_prev_hash")`	Remove the helper fingerprint columns.

`def save(df, table):`
`  (df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(table))`
`  n = spark.table(table).count()`
`  print(f"{table}: {n:,} rows")`
`  return n`
	save(): overwrite the table with the new data (the whole table is rebuilt), print and return the row count.

SCD2 shown with real data (customer 1)

"SCD2" means *Slowly Changing Dimension type 2*: **never overwrite** a customer; add a new version when something important changes, and remember when each version was valid.

silver.customer_history for customer 1

customer_id	version	valid_from	valid_to	is_current	name_masked	email_masked	email_hash	msisdn_masked	msisdn_hash	city	country	status	plan_id	_was_repaired
1	1	2026-09-01 08:00:00	2026-10-09 13:09:47.259	False	S*** d***	m***@example.nl	54d7451c6f5a	+316****80	3acc65c4e04b	Breda	NL	ACTIVE	8	True
1	2	2026-10-09 13:09:47.259	9999-12-31 00:00:00	True	S*** d***	m***@example.nl	54d7451c6f5a	+316****80	3acc65c4e04b	Breda	NL	ACTIVE	3	True

	What happened
version 1	From the creation (2026-09-01) the customer had plan 8.
version 2	On 2026-10-09 13:09 a change event arrived: plan 8 → 3 (and the phone in local 06-format, which auto-fix repaired). A tracked column changed, so a new version started; version 1 was closed at that moment.
why it matters	An invoice from September is matched to version 1 (plan 8); usage after 9 October to version 2 (plan 3). Without auto-fix, this event would have stayed in quarantine and Silver would still say plan 8.

Part 7 · Notebook 50: building the history

Job in one sentence: build `plan_history` and `customer_history` (one row per version) from the validated change events, with all personal data hashed or masked.

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
	Notebook header.

`# MAGIC %run /Workspace/Users/ahme63111@gmail.com/sentinel-dq-telecom/Silver_layer/49_silver_lib`
 	
`# COMMAND ----------`
	Load the Silver toolbox. Note: the Databricks copy loads `Silver_layer/49_silver_lib` (the copy in the sub-folder).

`# SILVER HISTORY (SCD2) with masked PII.`
`#   plan_history     : one row per plan version`
`#   customer_history : one row per customer version; email / phone / name are hashed and masked, never stored raw`
`# Input: only rows that passed the quality gate. Rebuilt from the ordered change events, so a replay gives the same`
`# result (idempotent). Surrogate keys, the unknown member and the star schema are built in Gold by dbt.`
 	
`# COMMAND ----------`
	Explanation: two history tables, personal data never stored raw, rebuilt from the ordered events so a replay gives the same result.

`# Remove the tables of the first Silver version: dimensions and facts now belong to Gold (dbt)`
`for t in OLD_TABLES:`
`    if spark.catalog.tableExists(t):`
`        spark.sql(f"DROP TABLE {t}")`
`        print("dropped", t)`
	Clean-up: drop the very old Silver dim_/fact_ tables if they still exist.

`plans = validated("crm_plans").filter("_op IN ('r', 'c', 'u')")`	Take the validated plan events, only snapshots (r), creates (c) and updates (u).
`p = scd2(plans, "plan_id", ["plan_name", "monthly_fee", "data_gb", "voice_minutes"])`	Build the plan history; a new version when the name, monthly fee, data GB or voice minutes change.

`plan_history = p.select("plan_id", "version", "valid_from", "valid_to", "is_current", "plan_name", "monthly_fee",`
`                         "data_gb", "voice_minutes", F.current_timestamp().alias("_loaded_at"))`
`save(plan_history, T_PLAN_HISTORY)`
	Choose the columns, add the load time, and save as `silver.plan_history`.

`cust = validated("crm_customers").filter("_op IN ('r', 'c', 'u')")`	Take the validated customer events (r, c, u). These include the repaired ones from notebook 22.
`cust = (cust.withColumn("msisdn_hash", hash_pii("msisdn"))` `             .withColumn("email_hash", hash_pii("email", lower=True)))`	Add two fingerprints: `msisdn_hash` (phone) and `email_hash` (e-mail, lower-case first).

`c2 = scd2(cust, "customer_id",`
`           ["msisdn_hash", "email_hash", "first_name", "last_name", "city", "country", "status", "plan_id"],`
`           created_col="created_at")`
	Build the customer history. A new version starts when the phone, e-mail, first or last name, city, country, status or plan changes. Version 1 starts at `created_at`.

`customer_history = c2.select(`
`    "customer_id", "version", "valid_from", "valid_to", "is_current",`
`    F.concat_ws(" ", mask_name("first_name"), mask_name("last_name")).alias("name_masked"),`
`    mask_email("email").alias("email_masked"), "email_hash",`
`    mask_msisdn("msisdn").alias("msisdn_masked"), "msisdn_hash",`
`    "city", "country", "status", "plan_id", "created_at",`
`    F.col("_dq_fixed").alias("_was_repaired"), F.current_timestamp().alias("_loaded_at"))`
	Choose the output columns. The name becomes `S*** d***`, the e-mail `m***@example.nl`, the phone `+316****80`, plus the two hashes. `_dq_fixed` is renamed `_was_repaired`. The raw name, e-mail and phone are **not** selected, so they never reach this table.

`save(customer_history, T_CUSTOMER_HISTORY)` `display(spark.table(T_CUSTOMER_HISTORY).orderBy("customer_id", "version").limit(20))`	Save as `silver.customer_history` and show 20 rows.

Output 1: `silver.plan_history` (20 rows today)

Column	Type	What it means (simple words)
`plan_id`	`int`	The plan's number.
`version`	`int`	1 for the first version, 2 when something changed, ...
`valid_from`	`timestamp`	When this version started.
`valid_to`	`timestamp`	When it ended (9999-12-31 = still valid).
`is_current`	`boolean`	true for the version that is valid now.
`plan_name`	`string`	Name, e.g. Basic S.
`monthly_fee`	`decimal(8,2)`	Price per month in euro.
`data_gb`	`int`	Data included (GB).
`voice_minutes`	`int`	Call minutes included.
`_loaded_at`	`timestamp`	When Silver was built.

Real rows

plan_id	version	valid_from	valid_to	is_current	plan_name	monthly_fee	data_gb	voice_minutes	_loaded_at
1	1	2026-10-09 13:06:44	9999-12-31 00:00:00	True	Basic S	10.00	2	100	2026-11-03 16:37:54
2	1	2026-10-09 13:06:44	9999-12-31 00:00:00	True	Basic M	12.00	5	200	2026-11-03 16:37:54
3	1	2026-10-09 13:06:44	9999-12-31 00:00:00	True	Basic L	15.00	8	300	2026-11-03 16:37:54

Output 2: silver.customer_history (979 rows today)

Column	Type	What it means (simple words)
`customer_id`	`bigint`	The customer's CRM number.
`version`	`int`	Version number (1, 2, ...).
`valid_from`	`timestamp`	Start of this version.
`valid_to`	`timestamp`	End of this version (9999-12-31 = current).
`is_current`	`boolean`	true = the version valid now; exactly one per customer.
`name_masked`	`string`	Name with only first letters: S*** d***.
`email_masked`	`string`	E-mail shown safely: m***@example.nl.
`email_hash`	`string`	Fingerprint of the e-mail (64 hex characters) to link without the real address.
`msisdn_masked`	`string`	Phone shown safely: +316****80.
`msisdn_hash`	`string`	Fingerprint of the phone: **the key used to link CRM and billing**.
`city`	`string`	City.
`country`	`string`	Country code.
`status`	`string`	ACTIVE, SUSPENDED or CHURNED.
`plan_id`	`int`	Plan at that time.
`created_at`	`timestamp`	When the customer was created.
`_was_repaired`	`boolean`	true = this version came from a repaired row (auto-fix or review app).
`_loaded_at`	`timestamp`	When Silver was built.

979 rows = 978 customers. Almost every customer has one version; customer 1 has two (the plan change shown in Part 6), so 978 + 1 = 979 rows. Exactly 978 rows have is_current = true.

Part 8 · Notebook 51: the clean tables

Job in one sentence: for every other dataset, keep only the **newest row per key**, swap the phone for its fingerprint, and save a simple clean table. No joining, no adding up: that is Gold's work.

> **Think of it like...**
> A photo album where you keep only the **latest** photo of each friend, and you put a sticker over their phone number.

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run /Workspace/Users/ahme63111@gmail.com/sentinel-dq-telecom/Silver_layer/49_silver_lib`
 	
`# COMMAND ----------`
	Notebook header; load the Silver toolbox (from `Silver_layer/49_silver_lib`).

`# SILVER CLEAN TABLES: one row per business key (newest version), only rows that passed the quality gate,`
`# phone numbers replaced by their hash. Nothing is joined or aggregated here: that is Gold (dbt).`
`#   CDC datasets (Postgres)  -> newest change event per key`
`#   file / stream datasets   -> one row per id (duplicates were already rejected by the gate)`
	Explanation: one row per key, only passed rows, phones hashed. For database change events: the newest event per key. For files: one row per id.

`OPS = "_op IN ('r', 'c', 'u')"` `LOADED = F.current_timestamp().alias("_loaded_at")`	`OPS` : only r/c/u events. `LOADED` : a column with the build time.
`def with_hash(df):` `    return df.withColumn("msisdn_hash", hash_pii("msisdn")).drop("msisdn")`	**with_hash**: add `msisdn_hash` and **remove** the raw `msisdn` column.
`acc = with_hash(latest_per_key(validated("billing_accounts").filter(OPS), ["account_id"]))` `save(acc.select("account_id", "account_ref", "msisdn_hash", "status", "created_at", "updated_at", LOADED), T_ACCOUNT)`	**account**: newest event per account_id, phone hashed → `silver.account`.

`inv = latest_per_key(validated("billing_invoices").filter(OPS), ["invoice_id"])`
`save(inv.select("invoice_id", "account_id", "period_start", "period_end", "total_amount", "status", "issued_at", LOADED),`
`     T_INVOICE)`
	invoice: newest event per invoice_id → `silver.invoice`.

`ln = latest_per_key(validated("billing_invoice_lines").filter(OPS), ["line_id"])` `save(ln.select("line_id", "invoice_id", "line_type", "description", "amount", LOADED), T_INVOICE_LINE)`	**invoice_line**: newest per line_id → `silver.invoice_line`.
`pay = latest_per_key(validated("billing_payments").filter(OPS), ["payment_id"])` `save(pay.select("payment_id", "invoice_id", "amount", "method", "status", "paid_at", LOADED), T_PAYMENT)`	**payment**: newest per payment_id → `silver.payment`.

`fi = with_hash(latest_per_key(validated("invoices_file"), ["invoice_id"]))`
`save(fi.select("invoice_id", "account_ref", "msisdn_hash", "period_start", "period_end", "plan_fee", "overage",`
`              "total_amount", "status", LOADED), T_INVOICE_FILE)`
	invoice_file (the monthly invoice CSV): one row per invoice_id, phone hashed.

`pf = latest_per_key(validated("payments_file"), ["payment_id"])` `save(pf.select("payment_id", "invoice_id", "amount", "method", "status", "paid_at", LOADED), T_PAYMENT_FILE)`	**payment_file** (the payment CSV): one row per payment_id.

`net = with_hash(latest_per_key(validated("network_cdr"), ["cdr_id"]))`
`save(net.select("cdr_id", "msisdn_hash", "event_type", "event_ts", "duration_sec", "data_mb", "cell_id", "call_result",`
`              LOADED), T_NETWORK_CDR)`
	network_cdr (call/SMS/data records): one row per cdr_id, phone hashed.

`# app events are optional: when their contract is disabled (contract/_app_events.yaml) the table is left as it is`
`if "app_events" in CONTRACTS:`
`    ev = latest_per_key(validated("app_events"), ["event_id"])`
`    save(ev.select("event_id", "customer_id", "event_type", "app_version", "event_ts", LOADED), T_APP_EVENT)`
`else:`
`    print("app_events contract disabled: silver.app_event not rebuilt")`
	app_event: only if the app_events contract is switched on. It is disabled now (the file is called `_app_events.yaml`), so the old table is left as it is instead of crashing.

`pu = with_hash(latest_per_key(validated("partner_usage"), ["partner_record_id"]))`
`save(pu.select("partner_record_id", "msisdn_hash", "service", "amount", "currency", "period_start", LOADED),`
`     T_PARTNER_USAGE)`
	partner_usage (roaming / international from partners): one row per record, phone hashed.

account (from billing_accounts): silver.account (994 rows today)

Column	Type	What it means (simple words)
`account_id`	`bigint`	Billing's own account number (different from the CRM customer_id).
`account_ref`	`string`	Account reference like ACC-00000001 (used by the CSV files).
`msisdn_hash`	`string`	Fingerprint of the phone (the raw phone is removed).
`status`	`string`	Status in the source (OPEN, PAID, ISSUED, COMPLETED...).
`created_at`	`timestamp`	When it was created in the source.
`updated_at`	`timestamp`	When it was last changed in the source.
`_loaded_at`	`timestamp`	When Silver was built.

Real rows

account_id	account_ref	msisdn_hash	status	created_at	updated_at
1	ACC-00000001	3acc65c4e04b9423	OPEN	2026-09-01 08:00:00	2026-09-01 08:00:00
2	ACC-00000002	935505aeb15fd3ac	OPEN	2026-09-01 08:00:00	2026-09-01 08:00:00

invoice (from billing_invoices): silver.invoice (3,000 rows today)

Column	Type	What it means (simple words)
`invoice_id`	`bigint`	Invoice number.
`account_id`	`bigint`	Billing's own account number (different from the CRM customer_id).
`period_start`	`date`	First day of the billed month.
`period_end`	`date`	Last day of the billed month.
`total_amount`	`decimal(10,2)`	Invoice total in euro.
`status`	`string`	Status in the source (OPEN, PAID, ISSUED, COMPLETED...).
`issued_at`	`timestamp`	When the invoice was issued.
`_loaded_at`	`timestamp`	When Silver was built.

Real rows

invoice_id	account_id	period_start	period_end	total_amount	status	issued_at	_loaded_at
1	1	2026-07-01	2026-07-31	32.00	ISSUED	2026-07-31 00:00:00	2026-11-03 16:38:33
2	1	2026-08-01	2026-08-31	43.15	ISSUED	2026-08-31 00:00:00	2026-11-03 16:38:33

invoice_line (from billing_invoice_lines): silver.invoice_line (3,886 rows today)

Column	Type	What it means (simple words)
`line_id`	`bigint`	Invoice line number.
`invoice_id`	`bigint`	Invoice number.
`line_type`	`string`	PLAN_FEE, DATA_OVERAGE, VOICE or ROAMING.
`description`	`string`	Text of the line.
`amount`	`decimal(10,2)`	Amount in euro (negative = credit or refund, flagged but accepted).
`_loaded_at`	`timestamp`	When Silver was built.

Real rows

line_id	invoice_id	line_type	description	amount	_loaded_at
1	1	PLAN_FEE	Monthly plan fee	32.00	2026-11-03 16:38:35
2	2	PLAN_FEE	Monthly plan fee	32.00	2026-11-03 16:38:35
3	2	DATA_OVERAGE	Data overage	11.15	2026-11-03 16:38:35

payment (from billing_payments): silver.payment (2,679 rows today)

Column	Type	What it means (simple words)
`payment_id`	`bigint`	Payment number.
`invoice_id`	`bigint`	Invoice number.
`amount`	`decimal(10,2)`	Amount in euro (negative = credit or refund, flagged but accepted).
`method`	`string`	CARD, SEPA or IDEAL.
`status`	`string`	Status in the source (OPEN, PAID, ISSUED, COMPLETED...).
`paid_at`	`timestamp`	When it was paid.
`_loaded_at`	`timestamp`	When Silver was built.

Real rows

payment_id	invoice_id	amount	method	status	paid_at	_loaded_at
1	1	32.00	IDEAL	PAID	2026-07-31 12:00:00	2026-11-03 16:38:38
2	2	43.15	IDEAL	PAID	2026-08-31 12:00:00	2026-11-03 16:38:38

invoice_file (from the invoice CSV): silver.invoice_file (2,952 rows today)

Column	Type	What it means (simple words)
`invoice_id`	`bigint`	Invoice number.
`account_ref`	`string`	Account reference like ACC-00000001 (used by the CSV files).
`msisdn_hash`	`string`	Fingerprint of the phone (the raw phone is removed).
`period_start`	`date`	First day of the billed month.
`period_end`	`date`	Last day of the billed month.
`plan_fee`	`decimal(10,2)`	Monthly plan fee on the file invoice.

Column	Type	What it means (simple words)
overage	decimal(10,2)	Extra usage charged.
total_amount	decimal(10,2)	Invoice total in euro.
status	string	Status in the source (OPEN, PAID, ISSUED, COMPLETED...).
_loaded_at	timestamp	When Silver was built.

Real rows

invoice_id	account_ref	msisdn_hash	period_start	period_end	plan_fee	overage	total_amount	status
1	ACC-00000001	3acc65c4e04b9423	2026-07-01	2026-07-31	32.00	0.00	32.00	ISSUED
2	ACC-00000001	3acc65c4e04b9423	2026-08-01	2026-08-31	32.00	11.15	43.15	ISSUED

payment_file (from the payment CSV): silver.payment_file (2,684 rows today)

Column	Type	What it means (simple words)
payment_id	bigint	Payment number.
invoice_id	bigint	Invoice number.
amount	decimal(10,2)	Amount in euro (negative = credit or refund, flagged but accepted).
method	string	CARD, SEPA or IDEAL.
status	string	Status in the source (OPEN, PAID, ISSUED, COMPLETED...).
paid_at	timestamp	When it was paid.
_loaded_at	timestamp	When Silver was built.

Real rows

payment_id	invoice_id	amount	method	status	paid_at	_loaded_at
1	1	32.00	IDEAL	PAID	2026-07-31 12:00:00	2026-11-03 16:38:44
2	2	43.15	IDEAL	PAID	2026-08-31 12:00:00	2026-11-03 16:38:44

network_cdr (from the network CSV files): silver.network_cdr (40,080 rows today)

Column	Type	What it means (simple words)
cdr_id	string	Call record id.
msisdn_hash	string	Fingerprint of the phone (the raw phone is removed).
event_type	string	VOICE, SMS or DATA.
event_ts	timestamp	When it happened.
duration_sec	int	Call length in seconds.
data_mb	decimal(12,2)	Data used in MB.
cell_id	string	Which mast (cell tower). Can be empty for one file that had no cell_id column (flagged).
call_result	string	COMPLETED or DROPPED.
_loaded_at	timestamp	When Silver was built.

Real rows

cdr_id	msisdn_hash	event_type	event_ts	duration_sec	data_mb	cell_id	call_result
CDR-20260928-000000	0f3106094d7f17cb	VOICE	2026-09-28 01:45:28	2195	0.00	CELL-033	COMPLETED
CDR-20260928-000001	434e97e9dd31c887	VOICE	2026-09-28 03:18:09	243	0.00	CELL-041	COMPLETED
CDR-20260928-000002	db5fb95042ca61f5	SMS	2026-09-28 01:48:19	0	0.00	CELL-019	COMPLETED

partner_usage (from the partner CSV): silver.partner_usage (583 rows today)

Column	Type	What it means (simple words)
partner_record_id	string	Partner's record id.
msisdn_hash	string	Fingerprint of the phone (the raw phone is removed).
service	string	ROAMING or INTERNATIONAL...
amount	decimal(10,2)	Amount in euro (negative = credit or refund, flagged but accepted).
currency	string	Currency, e.g. EUR.
period_start	date	First day of the billed month.
_loaded_at	timestamp	When Silver was built.

Real rows

partner_record_id	msisdn_hash	service	amount	currency	period_start
PRT-202607-00000	2c0cb4dfd8896cce	ROAMING	22.44	EUR	2026-07-01
PRT-202607-00001	3dfdfcc67307ca08	ROAMING	20.92	EUR	2026-07-01

validated versus clean

`silver.billing_payments_validated` can hold several events for the same payment (snapshot, then update) and all the quality columns. `silver.payment` holds **one row per payment**, the latest, with only business columns. Gold (dbt) reads the clean tables.

Part 9 · Notebook 52: linking CRM and billing

Job in one sentence: CRM and billing use **different numbers** for the same person, so link them through the **phone fingerprint** and say for every phone whether it matches.

> **Think of it like...**
> Two class lists from two teachers. One uses student numbers, the other uses seat numbers. The only thing both lists have is the child's birthday. You put the lists side by side and match by birthday: matched, only on list A, only on list B, or two children with the same birthday (unclear).

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run /Workspace/Users/ahme63111@gmail.com/sentinel-dq-telecom/Silver_layer/49_silver_lib`
 	
`# COMMAND ----------`
	Notebook header; load the Silver toolbox.

`# CUSTOMER_XREF: links the CRM customer id to the billing account id through the phone number.`
`# The ids differ by design (CRM customer_id is not the billing account_id), so the phone number is the only link.`
`# The phone number is compared as a hash, nothing raw is stored.`
`#   MATCHED      : one current customer and one account share the phone number`
`#   CRM_ONLY     : customer without a billing account   (reconciliation R1)`
`#   BILLING_ONLY : account without a customer           (reconciliation R2)`
`#   AMBIGUOUS    : the phone number belongs to more than one customer or account`
`# Run after 50 and 51.`
	Explanation of the four results: MATCHED, CRM_ONLY (reconciliation check R1), BILLING_ONLY (R2), AMBIGUOUS. Runs after 50 and 51.

`crm = (spark.table(T_CUSTOMER_HISTORY).filter("is_current")`
`.select("customer_id", "msisdn_hash", F.col("status").alias("crm_status"))`
`.withColumn("_crm_n", F.count("*").over(Window.partitionBy("msisdn_hash"))))`
	CRM side: the **current** version of each customer (number, phone hash, status). `_crm_n` = how many current customers share this phone hash.

`acc = (spark.table(T_ACCOUNT).select("account_id", "account_ref", "msisdn_hash", F.col("status").alias("billing_status"))` `.withColumn("_acc_n", F.count("*").over(Window.partitionBy("msisdn_hash"))))`	Billing side: each account (number, reference, phone hash, status). `_acc_n` = how many accounts share this phone hash.
`x = crm.alias("c").join(acc.alias("a"), F.col("c.msisdn_hash") == F.col("a.msisdn_hash"), "full_outer")`	Put both lists side by side by phone hash. **full_outer** keeps every customer and every account, even without a partner.

`status = (F.when(F.col("c.customer_id").isNull(), "BILLING_ONLY")`
`.when(F.col("a.account_id").isNull(), "CRM_ONLY")`
`.when((F.col("_crm_n") == 1) & (F.col("_acc_n") == 1), "MATCHED")`
`.otherwise("AMBIGUOUS"))`
	The verdict: no customer → BILLING_ONLY; no account → CRM_ONLY; exactly one of each → MATCHED; otherwise AMBIGUOUS.

`xref = x.select(F.coalesce(F.col("c.msisdn_hash"), F.col("a.msisdn_hash")).alias("msisdn_hash"),`
`F.col("c.customer_id"), F.col("a.account_id"), F.col("a.account_ref"),`
`F.col("c.crm_status"), F.col("a.billing_status"), status.alias("match_status"),`
`F.current_timestamp().alias("_loaded_at"))`
	Choose the columns: the phone hash, both ids, the account reference, both statuses, the verdict and the build time.

`save(xref, T_XREF)` `display(spark.table(T_XREF).groupBy("match_status").count().orderBy("match_status"))`	Save as `silver.customer_xref` and show how many of each verdict.

Output: silver.customer_xref (1,009 rows today)

Column	Type	What it means (simple words)
`msisdn_hash`	`string`	Phone fingerprint: the only shared key between the two systems.
`customer_id`	`bigint`	CRM customer (empty for BILLING_ONLY).
`account_id`	`bigint`	Billing account (empty for CRM_ONLY).
`account_ref`	`string`	Account reference (ACC-...).
`crm_status`	`string`	Customer status in CRM.
`billing_status`	`string`	Account status in billing.
`match_status`	`string`	MATCHED, CRM_ONLY, BILLING_ONLY or AMBIGUOUS.
`_loaded_at`	`timestamp`	When Silver was built.

One real row of each verdict

msisdn_hash	customer_id	account_id	account_ref	crm_status	billing_status	match_status
1b8a553fc2e5783c	null	11	ACC-00000011	null	OPEN	BILLING_ONLY
8443990fb04737ba	1016	null	null	ACTIVE	null	CRM_ONLY
3acc65c4e04b9423	1	1	ACC-00000001	ACTIVE	OPEN	MATCHED

Verdict	Rows	What it means	Where it comes from
MATCHED	963	Customer and account found each other.	The normal case.

Verdict	Rows	What it means	Where it comes from
CRM_ONLY	15	A customer without a billing account: they may use the network but get no invoice.	The 15 test customers called “Orphan Crm” that were added on purpose.
BILLING_ONLY	31	An account without a known customer: invoices go to someone we cannot identify.	10 test accounts ACC-X1…X10 added on purpose, plus 21 accounts whose customer is still in quarantine (bad phone, status UNKNOWN, future date) and so is not in Silver yet.

Check: accounts in Silver 994 = 963 + 31; current customers in Silver 978 = 963 + 15. When one of the 21 quarantined customers is repaired, its row moves from BILLING_ONLY to MATCHED on the next run.

Part 10 · Notebook 53: checking Silver itself

Job in one sentence: after Silver is built, run a list of self-tests and print PASS, WARN or FAIL. It never changes data.

The code (exactly as it runs in Databricks)	In plain words

`# Databricks notebook source`
`# /// script`
`# [tool.databricks.environment]`
`# environment_version = "6"`
`# ///`
`# MAGIC %run /Workspace/Users/ahme63111@gmail.com/sentinel-dq-telecom/Silver_layer/49_silver_lib`
 	
`# COMMAND ----------`
	Notebook header; load the Silver toolbox.

`# VERIFY the Silver layer. Run it after 50, 51 and 52. Nothing here changes data.`
`checks = []`
`def check(area, name, ok, detail="", warn=False):`
`    checks.append((area, name, "PASS" if ok else ("WARN" if warn else "FAIL"), str(detail)))`
	`checks` collects the results. `check()` adds one line: PASS if ok, else WARN (when it is only a warning) or FAIL.

`def q(sql):` `    return spark.sql(sql).collect()[0][0]`	`q()` : run a SQL question and return its single answer.

`for t in SILVER_TABLES:`
`    ex = spark.catalog.tableExists(t)`
`    check("tables", f"{t.split('.')[-1]} exists and has rows", ex and spark.table(t).count() > 0,`
`          spark.table(t).count() if ex else "missing")`
	Every Silver table exists and has rows.

`old = [t.split('.')[-1] for t in OLD_TABLES if spark.catalog.tableExists(t)]` `check("tables", "old dim_* / fact_* tables are gone from Silver (they belong to Gold)", not old, old)`	The very old dim_/fact_ tables are gone.

`# 1) SCD2 shape`
`dc, dp = T_CUSTOMER_HISTORY, T_PLAN_HISTORY`
`dup_cur = q(f"SELECT count(*) FROM (SELECT customer_id FROM {dc} WHERE is_current GROUP BY 1 HAVING count(*) > 1)")`
`check("scd2", "customer_history: exactly one current row per customer", dup_cur == 0, f"{dup_cur} customers with 2+ current rows")`
	SCD2 shape 1: exactly one current row per customer.

`overlap = q(f"""SELECT count(*) FROM {dc} a JOIN {dc} b ON a.customer_id = b.customer_id AND a.version < b.version`
`                AND a.valid_to > b.valid_from""")`
`check("scd2", "customer_history: versions of a customer never overlap", overlap == 0, overlap)`
	SCD2 shape 2: two versions of one customer never overlap in time.

`bad_range = q(f"SELECT count(*) FROM {dc} WHERE valid_from >= valid_to")` `check("scd2", "customer_history: valid_from < valid_to", bad_range == 0, bad_range)`	**SCD2 shape 3**: every version starts before it ends.
`dup_ver = q(f"SELECT count(*) FROM (SELECT customer_id, version FROM {dc} GROUP BY 1, 2 HAVING count(*) > 1)")` `check("scd2", "customer_history: (customer_id, version) is unique", dup_ver == 0, dup_ver)`	**SCD2 shape 4**: (customer_id, version) is unique.
`dup_plan = q(f"SELECT count(*) FROM (SELECT plan_id FROM {dp} WHERE is_current GROUP BY 1 HAVING count(*) > 1)")` `check("scd2", "plan_history: exactly one current row per plan", dup_plan == 0, dup_plan)`	Plans: exactly one current row per plan.

`# 2) every customer / plan that passed the gate is in the history`
`v_cust = validated("crm_customers").filter("_op IN ('r','c','u')").select("customer_id").distinct().count()`
`d_cust = q(f"SELECT count(DISTINCT customer_id) FROM {dc}")`
`check("coverage", "customer_history covers every customer that passed the quality gate", v_cust == d_cust, f"validated {v_cust:,}; history {d_cust:,}")`
	Coverage: every customer that passed the gate is in the history.

`v_plan = validated("crm_plans").filter("_op IN ('r','c','u')").select("plan_id").distinct().count()`
`d_plan = q(f"SELECT count(DISTINCT plan_id) FROM {dp}")`
`check("coverage", "plan_history covers every plan that passed the quality gate", v_plan == d_plan, f"validated {v_plan:,}; history {d_plan:,}")`
	Every plan that passed is in the plan history.

`# 3) PII: no raw email, phone number or name in any Silver table built here`
`raw_pii = {"email", "msisdn", "first_name", "last_name"}`
`leaks = [(t.split('.')[-1], c) for t in SILVER_TABLES for c in spark.table(t).columns if c in raw_pii]`
`check("pii", "no raw PII column in the Silver tables", not leaks, leaks)`
	Privacy: no Silver table built here has a column called email, msisdn, first_name or last_name.

`hashed = q(f"SELECT count(*) FROM {dc} WHERE msisdn_hash IS NULL OR length(msisdn_hash) <> 64")` `check("pii", "customer_history: every phone number is hashed", hashed == 0, hashed)`	Every phone in customer_history is a 64-character hash.

`for t, col in [(T_NETWORK_CDR, "msisdn_hash"), (T_PARTNER_USAGE, "msisdn_hash"), (T_INVOICE_FILE, "msisdn_hash"), (T_ACCOUNT, "msisdn_hash")]:`
`    bad = q(f"SELECT count(*) FROM {t} WHERE {col} IS NULL OR length({col}) <> 64")`
`    check("pii", f"{t.split('.')[-1]}: every phone number is hashed", bad == 0, bad)`
	The same for network_cdr, partner_usage, invoice_file and account.

`# 4) clean tables: one row per key, row count = distinct keys that passed the gate`
`def clean_check(table, key, dataset, label, cdc=False):`
`    t = spark.table(table); n = t.count()`
`    v = validated(dataset)`
`    if cdc:`
`        v = v.filter("_op IN ('r','c','u')")`
`    expect = v.select(*key).distinct().count()`
`    check("clean", f"{label}: one row per key", n == t.select(*key).distinct().count(), f"{n:,} rows")`
`    check("clean", f"{label}: row count = distinct keys that passed the gate", n == expect, f"silver {n:,}; validated {expect:,}")`
	clean_check(): a clean table has one row per key, and as many rows as there are distinct keys that passed the gate.

The code (exactly as it runs in Databricks)	In plain words

`clean_check(T_ACCOUNT, ["account_id"], "billing_accounts", "account", True)`
`clean_check(T_INVOICE, ["invoice_id"], "billing_invoices", "invoice", True)`
`clean_check(T_INVOICE_LINE, ["line_id"], "billing_invoice_lines", "invoice_line", True)`
`clean_check(T_PAYMENT, ["payment_id"], "billing_payments", "payment", True)`
`clean_check(T_INVOICE_FILE, ["invoice_id"], "invoices_file", "invoice_file")`
`clean_check(T_PAYMENT_FILE, ["payment_id"], "payments_file", "payment_file")`
`clean_check(T_NETWORK_CDR, ["cdr_id"], "network_cdr", "network_cdr")`
`clean_check(T_APP_EVENT, ["event_id"], "app_events", "app_event")`
`clean_check(T_PARTNER_USAGE, ["partner_record_id"], "partner_usage", "partner_usage")`
	Run clean_check for all nine clean tables.

`# 5) customer_xref`
`x = {r[0]: r[1] for r in spark.sql(f"SELECT match_status, count(*) FROM {T_XREF} GROUP BY 1").collect()}`
`check("xref", "xref has MATCHED rows", x.get("MATCHED", 0) > 0, x)`
	xref: there are MATCHED rows.

`check("xref", "customers without a billing account are visible (CRM_ONLY, injected: 15)", x.get("CRM_ONLY", 0) > 0, x.get("CRM_ONLY", 0), warn=True)` `check("xref", "accounts without a customer are visible (BILLING_ONLY, injected: 10)", x.get("BILLING_ONLY", 0) > 0, x.get("BILLING_ONLY", 0), warn=True)`	CRM_ONLY and BILLING_ONLY rows are visible (the injected test problems). Only a warning.
`dup_c = q(f"SELECT count(*) FROM (SELECT customer_id FROM {T_XREF} WHERE customer_id IS NOT NULL GROUP BY 1 HAVING count(*) > 1)")` `check("xref", "a customer appears once in the xref", dup_c == 0, dup_c, warn=True)`	A customer appears only once in the xref (warning).

`out = spark.createDataFrame(checks, "area string, `check` string, status string, detail string")`
`display(out.orderBy(F.when(F.col("status") == "FAIL", 0).when(F.col("status") == "WARN", 1).otherwise(2), "area"))`
`n_fail = sum(1 for c in checks if c[2] == "FAIL"); n_warn = sum(1 for c in checks if c[2] == "WARN")`
`print(f"\n{len(checks)} checks: {len(checks) - n_fail - n_warn} PASS, {n_warn} WARN, {n_fail} FAIL")`
`print("SILVER OK" if n_fail == 0 else "SILVER HAS FAILURES - fix the FAIL rows above")`
	Show all checks (FAIL first), count them and print “SILVER OK” or “SILVER HAS FAILURES”.

Watch out

The privacy check covers the Silver tables built by 50–52 (`SILVER_TABLES`). The `*_validated` tables are not in that list and do contain raw phone numbers and e-mails; they are an internal step and Gold reads only the hashed/masked tables (dbt test `assert_no_raw_pii_in_gold` proves it).

Part 11 · How every output connects

Table to table

quarantine.<ds> status, _fixed	21 fills _fixed → 22 reads _fixed	**silver.<ds>_validated** good + repaired rows
silver.crm_customers_validated **silver.crm_plans_validated**	50 →	**customer_history** **plan_history**
other *_validated	51 →	**account · invoice · invoice_line · payment** **invoice_file · payment_file · network_cdr · partner_usage**
customer_history + account	52 →	**customer_xref**
21 · 22 · 30	append →	**monitoring.dq_results** **(30 also → monitoring.incidents)**
all Silver	53 reads →	**PASS / WARN / FAIL report**

Column to column (the phone number's journey)

Table	Column	Value for customer 1
Bronze change event	`after.msisdn`	`0660****680` (wrong local format)
quarantine.crm_customers	`msisdn` / `_fixed['msisdn']`	`0660****680` / `+3166****680`
silver.crm_customers_validated	`msisdn` (+ `_dq_fixed = true`)	`+3166****680`
silver.customer_history	`msisdn_hash` / `msisdn_masked`	`3acc65c4e04b…` / `+316****80`
silver.customer_xref	`msisdn_hash` → `match_status`	`3acc65c4e04b…` → MATCHED with account 1 (ACC-00000001)

One customer through every table (real rows)

1 · quarantine.crm_customers (after 21 and 22)

customer_id	msisdn	fixed_msisdn	status	_failed_rules	_failure_reasons	_quarantine_status	_attempts	_row_id	_op	q_at
1	0660****680	+3166****680	ACTIVE	['CUST_MSISDN_FORMAT']	CUST_MSISDN_FORMAT	REVALIDATED_OK	1	telecom.crm.customers|0|1030	u	2026-10-10 18:45:39.191152

2 · silver.crm_customers_validated (added by 22)

customer_id	msisdn	city	status	plan_id	_row_id	_op	_is_latest	_flags	_dq_fixed	_contract_version
1	+3166****680	Breda	ACTIVE	3	telecom.crm.customers|0|1030	u	True	[]	True	1.0

3 · silver.customer_history (built by 50)

customer_id	version	valid_from	valid_to	is_current	name_masked	email_masked	email_hash	msisdn_masked	msisdn_hash	city	country	status	plan_id	_was_repaired
1	1	2026-09-01 08:00:00	2026-10-09 13:09:47.259	False	S*** d***	m***@example.nl	54d7451c6f5a	+316****80	3acc65c4e04b	Breda	NL	ACTIVE	8	True
1	2	2026-10-09 13:09:47.259	9999-12-31 00:00:00	True	S*** d***	m***@example.nl	54d7451c6f5a	+316****80	3acc65c4e04b	Breda	NL	ACTIVE	3	True

4 · silver.customer_xref (built by 52)

msisdn_hash	customer_id	account_id	account_ref	crm_status	billing_status	match_status
3acc65c4e04b	1	1	ACC-00000001	ACTIVE	OPEN	MATCHED

The numbers add up

Check	Numbers	OK?
Every Bronze customer row is either good or in quarantine	1,031 = 979 validated + 52 rejected	✅
Validated = passed at the gate + repaired	979 = 940 + 39	✅
History covers every validated customer	978 distinct customer_ids in both tables (979 versions)	✅
Accounts are either matched or alone	994 = 963 MATCHED + 31 BILLING_ONLY	✅
Current customers are either matched or alone	978 = 963 MATCHED + 15 CRM_ONLY	✅

Part 12 · Cheat sheet and questions

Notebook	One-line job	Output tables
21_quarantine_autofix	Repair PENDING_FIX rows with the contract's fixes.	quarantine.<ds> (updated), monitoring.dq_results
22_revalidate	Re-test repaired rows; pass → validated, fail → REJECTED.	silver.<ds>_validated (rows added), quarantine.<ds> (updated), monitoring.dq_results
30_observability_monitors	Freshness, volume, distribution, schema, lineage.	monitoring.dq_results, monitoring.incidents
49_silver_lib	Tools: names, salt, latest row, hash, mask, SCD2, save.	—
50_silver_history	Versions of customers and plans, PII hidden.	silver.customer_history, silver.plan_history
51_silver_clean	Newest row per key, phone hashed.	silver.account, invoice, invoice_line, payment, invoice_file, payment_file, network_cdr, partner_usage (app_event)
52_silver_customer_xref	Link CRM ↔ billing by phone hash.	silver.customer_xref
53_silver_verify	Self-tests of Silver.	— (report on screen)

Questions a beginner asks

Why not just delete bad rows?
Then nobody would know they were wrong, totals would silently shrink, and nobody could fix them. Quarantine keeps them visible with the reason.

Why not fix everything automatically?
Only when the right answer is certain (06… → +316…). A phone like ABC has no known right answer; inventing one would be lying.

Can a person push a wrong value into Silver with the review app?
No. The edit goes to _fixed and is re-tested by notebook 22 with the same rules.

Why rebuild Silver every run instead of updating it?
Rebuilding from the events always gives the same answer (idempotent). A replay or a bug fix cannot leave half-updated rows behind.

Why hash the phone and not delete it?
We still need to link systems and count per customer. The hash allows linking without anyone seeing the real number.

What closes an incident?
The data. When the check passes on a later run, sync_incidents marks it RESOLVED with that run id.

What does _dq_fixed = true tell me?
That this row only made it after a repair. Useful to audit how much data depended on fixes.

Things to know about the current setup

- The Silver notebooks in Databricks load the helper from the sub-folder copy `Silver_layer/49_silver_lib`; keep that copy (or change the path) when tidying the workspace.
- Kafka row ids now include the Kafka timestamp (fix of 3 November 2026), so quarantine row ids change once on the next run; statuses are rebuilt by 20 → 21 → 22.
- All numbers in this book were read from Databricks on 3 November 2026.